

A.B.V.M. Agrawal Jatiya Kosh's

Ghanshyamdas Jalan College of Science & Commerce

Malad (East), Mumbai

(Affiliated to University of Mumbai)

DEPARTMENT OF INFORMATION TECHNOLOGY

Applied Business Analytics Practical

B.Sc. Information Technology, Semester V

Submitted by

Ayesha Sarguru

Seat Number:502520019

Under the Guidance of

Prof. Raviprakash Jaiswal

Academic Year 2026–27

October 2026



Ghanshyamdas Jalan College of Science & Commerce

Malad (East), Mumbai

(Affiliated to University of Mumbai)

Certificate by the Institute

Department of Information Technology

I hereby certify that Ms. **Ayesha Sarguru**, Roll Number 20, a student of Ghanshyamdas Jalan College of Science & Commerce, Malad (East), studying in **B.Sc. Information Technology**, has successfully completed the practical work in **Applied Business Analytics** for the academic year 2026–27, under the guidance of Prof. Raviprakash Jaiswal. To the best of my knowledge, the work of the student is original and the information included in the practical is correct.

Internal Guide	**External Examiner**	**Head of the Department**

Date: _____ **College Seal:**

INDEX

Sr. No.	Title	Page No.
1	To collect business data and perform Exploratory Data Analysis (EDA) to understand the structure, quality and patterns of a sales dataset.	1
2	To analyze customer data to measure total revenue, product-wise and city-wise sales, spending patterns and customer segments for business decision-making.	7
3	To calculate descriptive statistics (mean, median, mode, variance, standard deviation and range) for a business variable of interest.	11
4	To visualize the distribution of a business variable using histograms, box plots and violin plots and to identify outliers.	13
5	To perform correlation analysis to determine the strength and direction of relationships between advertising channels and sales revenue.	17
6	To apply linear regression analysis to identify the relationship between TV advertising expenditure (independent variable) and sales revenue (dependent variable).	20
7	To utilize a Business Intelligence (BI) tool such as Tableau to extract meaningful insights from a dataset and create interactive visualizations for effective business decision-making.	23
8	To simulate a probability experiment, such as rolling a die, using Python and calculate the probabilities of different outcomes.	24
9	To generate random numbers from Normal, Uniform, and Exponential probability distributions using Python and analyze their properties.	25
10	To develop a decision tree model for business decisions under uncertainty and apply Expected Monetary Value (EMV) to select the optimal alternative.	28
11	To conduct a survey, collect data from a sample population using appropriate sampling techniques, and summarize the collected data for business analysis.	30
12	To apply the Central Limit Theorem (CLT) to analyze the sampling distribution of a sample mean and estimate population parameters.	32
13	To calculate confidence intervals for population means and proportions using sample data and interpret the results in a business context.	34
14	To apply bootstrapping to estimate confidence intervals for non-parametric statistics such as the median.	36
15	To formulate null and alternative hypotheses, perform an appropriate statistical hypothesis test using sample data, and interpret the results in a business context.	38
16	To perform A/B testing on a website or marketing campaign to evaluate the effectiveness of two different strategies and make a data-driven business decision.	39

Sr. No.	Title	Page No.
17	To develop a regression model to predict future sales based on historical data, assess the performance of the model, and interpret the significance of predictor variables.	41
18	To apply time series forecasting techniques, such as exponential smoothing and ARIMA, to forecast future product demand and evaluate forecast accuracy.	44

Practical 1A: Exploratory Data Analysis (EDA)

Aim: To collect business data and perform Exploratory Data Analysis (EDA) to understand the structure, quality and patterns of a sales dataset.

Theory: *Exploratory Data Analysis (EDA) examines and visualizes a dataset to understand its structure, detect missing values, duplicates and outliers, and discover patterns before deeper analysis. Python's Pandas, Matplotlib and Seaborn libraries are used on the retail sales data in sales.csv.*

Pract1a.py

```
# Step 1: Import Required Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
# to set Max Rows to display
# pd.set_option('display.max_rows', 500)

# Step 2: Load Dataset
df = pd.read_csv("sales.csv")
print("Display all Records in string Format")
print(df.to string())
# Step 3: Display First Five Records
print("Display First Five Records")
print(df.head())
print("Display last 10 Records")
print(df.tail(10))

# Step 4: Check Dataset Information
print("Display Dataset Information")
print(df.info())

# Step 5: Statistical Summary
print("Display Statistical Summary:")
print(df.describe())

#Step 6: Check Missing Values
print("Check Missing Values:")
print(df.isnull().sum())

# Step 7: Check Duplicate Records
print("Check Duplicate Records:")
print(df.duplicated().sum())

# Step 8: Check Product Distribution
print("Display Product Distribution:")
print(df["Product"].value_counts())


sns.countplot(data=df,x="Product")
plt.xticks(rotation=270)
plt.title("Product Distribution || Ayesha Sarguru")
plt.show()

# Step 9: Gender Distribution
sns.countplot(data=df,x="Gender")
plt.title("Gender Distribution || Ayesha Sarguru")
plt.show()


# Step 10: City-wise Customerss
sns.countplot(data=df,x="City")
plt.xticks(rotation=45)
plt.title("City-wise Customers || Ayesha Sarguru")
plt.show()

# Step 11: Price Distribution
plt.hist(df["Price"],bins=4)
plt.xlabel("Price")
plt.ylabel("Frequency")
plt.title("Price Distribution || Ayesha Sarguru")
plt.show()
```

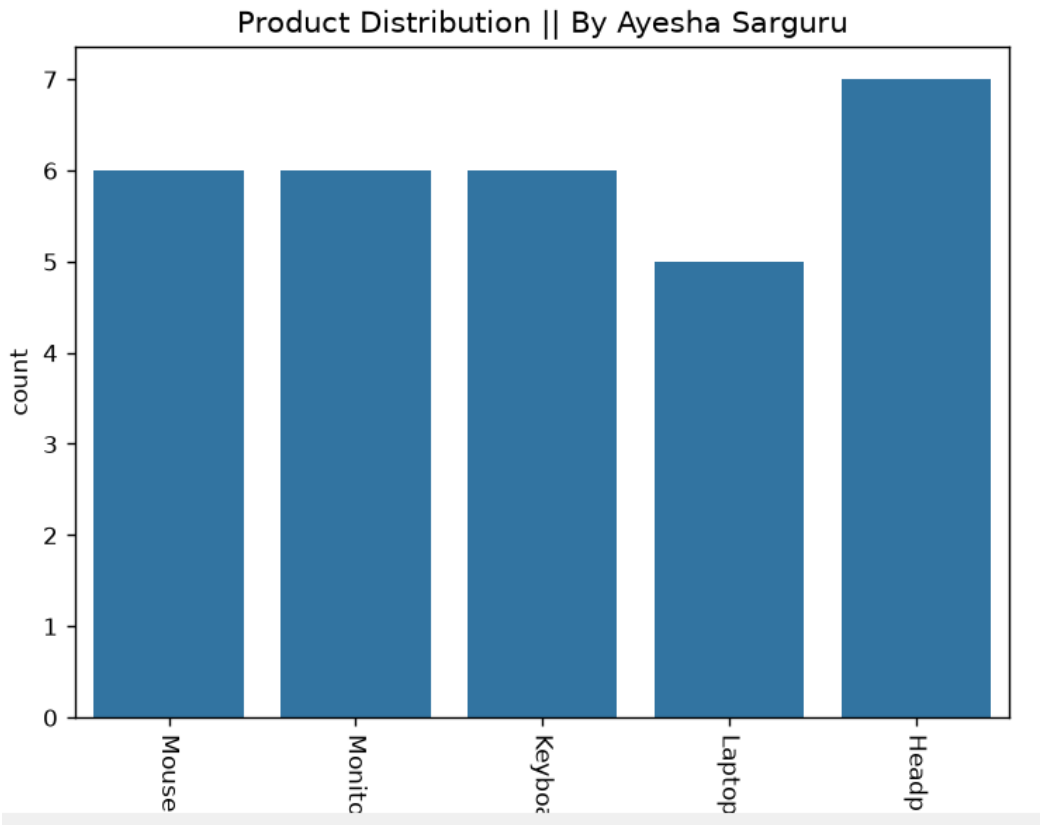
```
# Step 12: Box Plot
sns.boxplot(y=df["Total"])
plt.title("Box Plot of Total sales || Ayesha Sarguru")
plt.show()

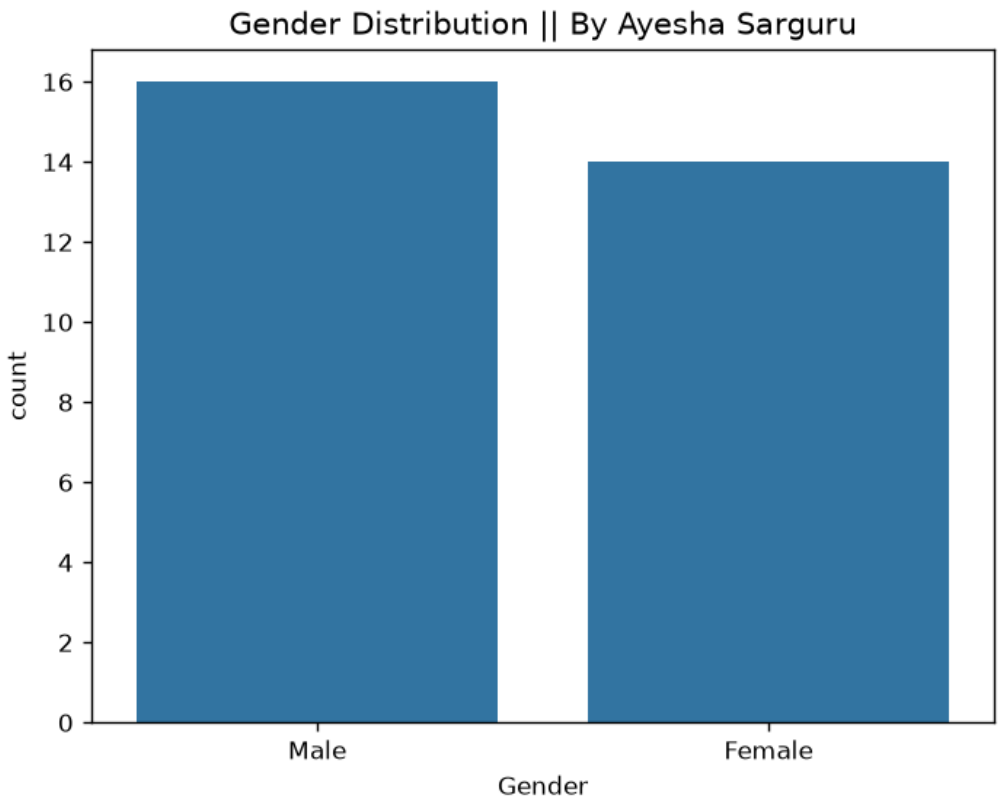
# Step 13: Correlation Heatmap
sns.heatmap(df.corr(numeric_only=True),annot=True,cmap= "rocket")
plt.title("Correlation Heatmap || Ayesha Sarguru")
plt.show()
```

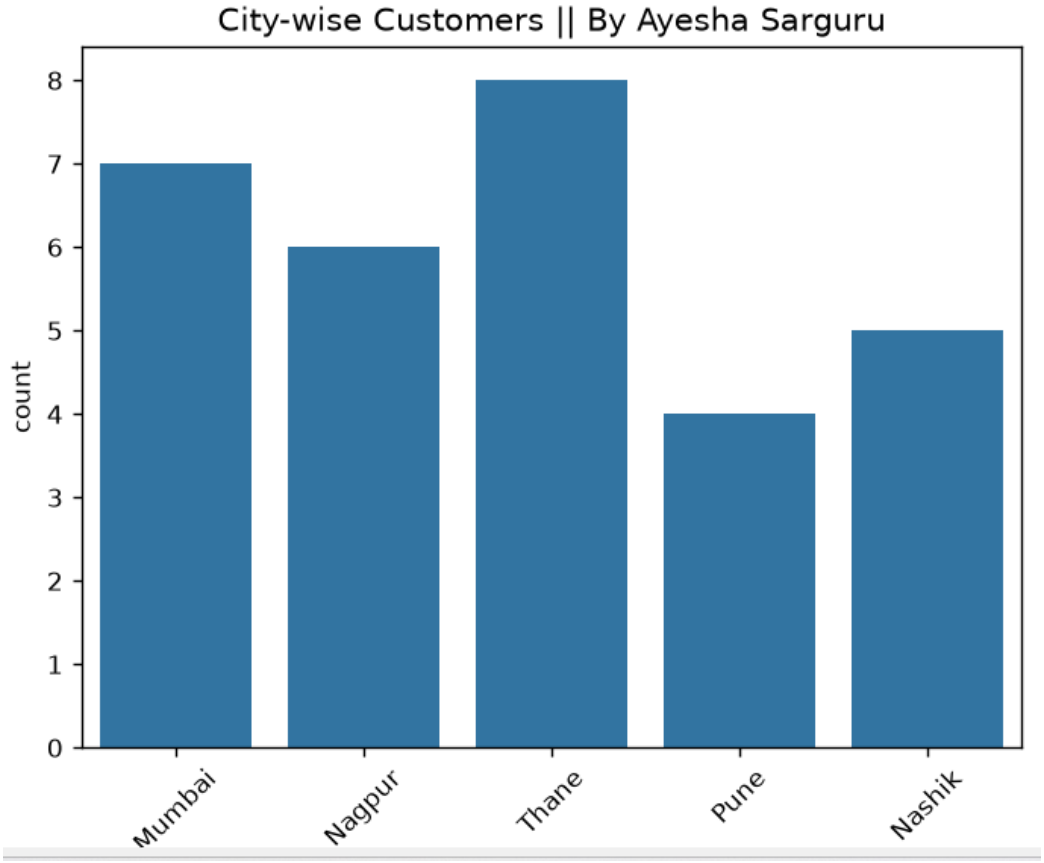
Output:

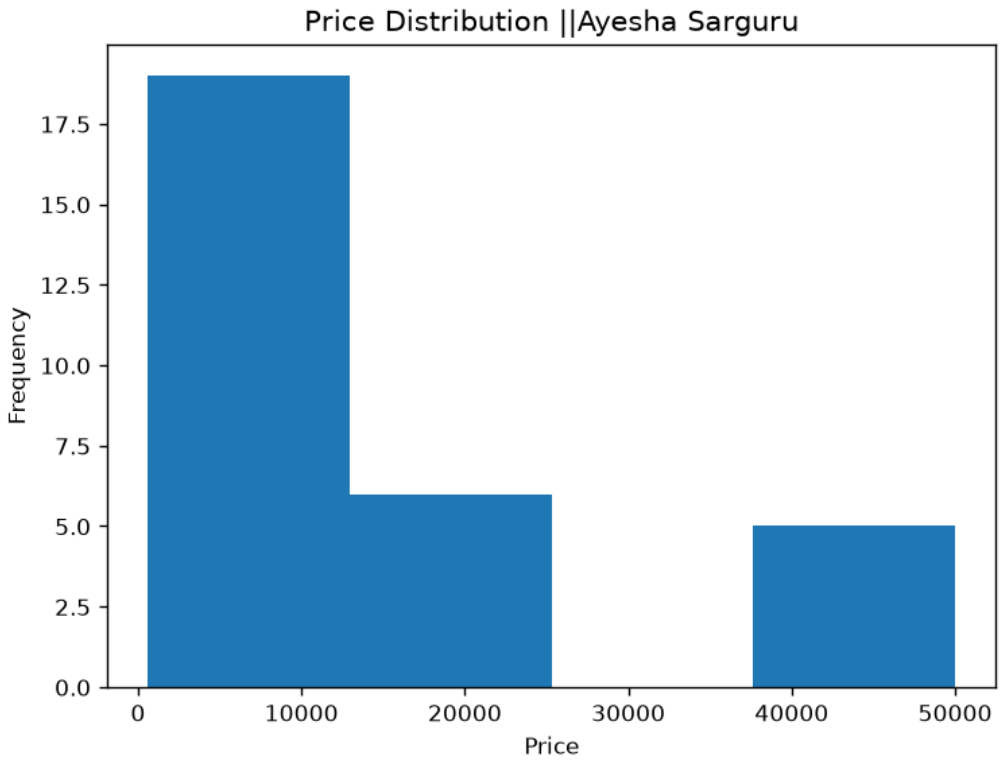
```
PS D:\business analytics> & C:\Users\Student\AppData\Local\Microsoft\WindowsApps\pytho
Display all Records in string Format
    CustomerID  Gender  Age    City     Product  Quantity  Price   Total        Date
0          101    Male   28  Mumbai       Mouse         3    600    1800  02-01-2026
1          102  Female   40  Nagpur     Monitor         2  15000   30000  03-01-2026
2          103  Female   20  Nagpur       Mouse         1    600     600  04-01-2026
3          104    Male   42  Nagpur     Monitor         3  15000   45000  05-01-2026
4          105    Male   38  Mumbai    Keyboard         3   1200    3600  06-01-2026
5          106    Male   20   Thane    Keyboard         1   1200    1200  07-01-2026
6          107    Male   33  Mumbai      Laptop         2  50000  100000  08-01-2026
7          108  Female   50  Nagpur  Headphones         1   2500    2500  09-01-2026
8          109  Female   27    Pune  Headphones         1   2500    2500  10-01-2026
9          110    Male   33   Thane     Monitor         2  15000   30000  11-01-2026
10         111  Female   23  Nashik      Laptop         1  50000   50000  12-01-2026
11         112    Male   29  Nashik  Headphones         2   2500    5000  13-01-2026
12         113  Female   38  Mumbai  Headphones         2   2500    5000  14-01-2026
13         114  Female   33    Pune     Monitor         1  15000   15000  15-01-2026
14         115  Female   22  Nagpur    Keyboard         3   1200    3600  16-01-2026
15         116    Male   36  Nagpur  Headphones         1   2500    2500  17-01-2026
16         117    Male   35  Mumbai    Keyboard         2   1200    2400  18-01-2026
17         118  Female   40    Pune    Keyboard         3   1200    3600  19-01-2026
18         119    Male   20    Pune       Mouse         3    600    1800  20-01-2026
19         120    Male   32   Thane  Headphones         3   2500    7500  21-01-2026
20         121  Female   34  Nashik    Keyboard         3   1200    3600  22-01-2026
21         122    Male   32   Thane  Headphones         3   2500    7500  23-01-2026
22         123    Male   33  Mumbai       Mouse         3    600    1800  24-01-2026
23         124    Male   50   Thane     Monitor         2  15000   30000  25-01-2026
24         125  Female   33  Nashik     Monitor         2  15000   30000  26-01-2026
25         126  Female   34   Thane      Laptop         3  50000  150000  27-01-2026
26         127    Male   37   Thane      Laptop         1  50000   50000  28-01-2026
27         128  Female   21  Mumbai       Mouse         1    600     600  29-01-2026
28         129  Female   20  Nashik      Laptop         1  50000   50000  30-01-2026
29         130    Male   45   Thane       Mouse         2    600    1200  31-01-2026
Display First Five Records
   CustomerID  Gender  Age    City   Product  Quantity  Price  Total        Date
0         101    Male   28  Mumbai     Mouse         3    600   1800  02-01-2026
1         102  Female   40  Nagpur   Monitor         2  15000  30000  03-01-2026
2         103  Female   20  Nagpur     Mouse         1    600    600  04-01-2026
3         104    Male   42  Nagpur   Monitor         3  15000  45000  05-01-2026
4         105    Male   38  Mumbai  Keyboard         3   1200   3600  06-01-2026
Display last 10 Records
    CustomerID  Gender  Age    City     Product  Quantity  Price   Total        Date
20         121  Female   34  Nashik    Keyboard         3   1200    3600  22-01-2026
21         122    Male   32   Thane  Headphones         3   2500    7500  23-01-2026
22         123    Male   33  Mumbai       Mouse         3    600    1800  24-01-2026
23         124    Male   50   Thane     Monitor         2  15000   30000  25-01-2026
```

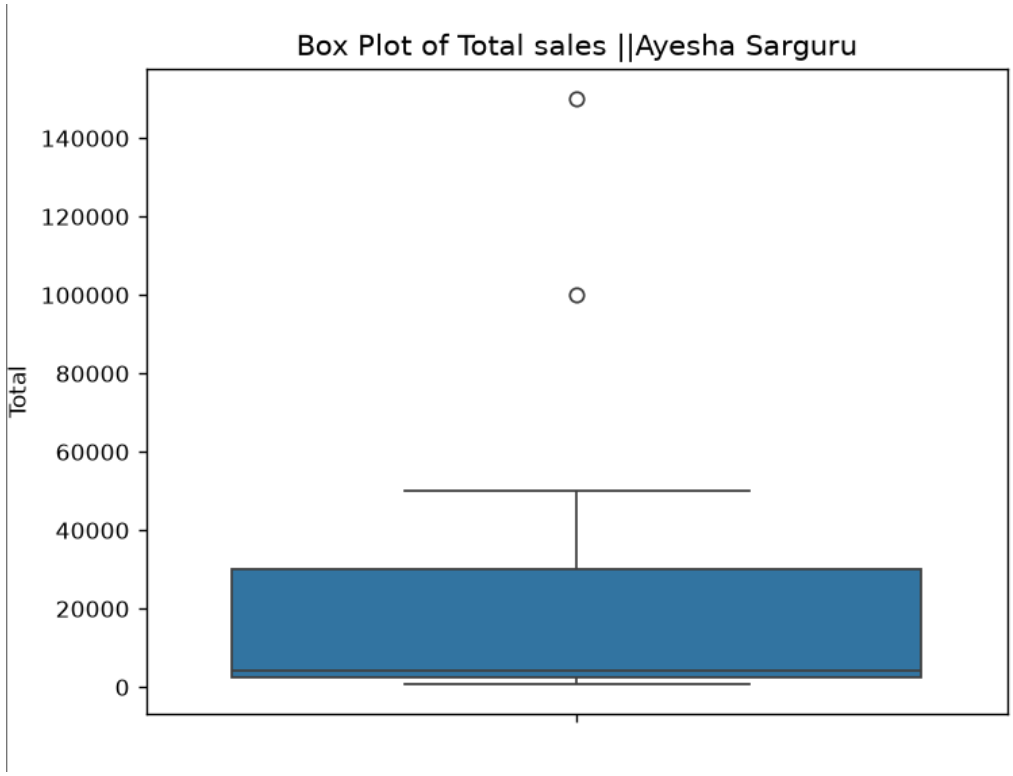
```
PS D:\business analytics> & C:\Users\Student\AppData\Local\Microsoft\WindowsApps\pyt
25         126  Female   34   Thane      Laptop         3  50000  150000  27-01-2026
26         127    Male   37   Thane      Laptop         1  50000   50000  28-01-2026
27         128  Female   21  Mumbai       Mouse         1    600     600  29-01-2026
28         129  Female   20  Nashik      Laptop         1  50000   50000  30-01-2026
29         130    Male   45   Thane       Mouse         2    600    1200  31-01-2026
Display Dataset Information
<class 'pandas.DataFrame'>
RangeIndex: 30 entries, 0 to 29
Data columns (total 9 columns):
 #   Column      Non-Null Count  Dtype
---  ------      --------------  -----
 0   CustomerID  30 non-null     int64
 1   Gender      30 non-null     str
 2   Age         30 non-null     int64
 3   City        30 non-null     str
 4   Product     30 non-null     str
 5   Quantity    30 non-null     int64
 6   Price       30 non-null     int64
 7   Total       30 non-null     int64
 8   Date        30 non-null     str
dtypes: int64(5), str(4)
memory usage: 2.2 KB
None
Display Statistical Summary:
       CustomerID        Age   Quantity         Price          Total
count   30.000000  30.000000  30.000000     30.000000      30.000000
mean   115.500000  32.600000   2.033333  12276.666667   21276.666667
std      8.803408   8.503954   0.850287  17987.595023   33487.657874
min    101.000000  20.000000   1.000000    600.000000     600.000000
25%    108.250000  27.250000   1.000000   1200.000000    2425.000000
50%    115.500000  33.000000   2.000000   2500.000000    4300.000000
75%    122.750000  37.750000   3.000000  15000.000000   30000.000000
max    130.000000  50.000000   3.000000  50000.000000  150000.000000
Check Missing Values:
CustomerID    0
Gender        0
Age           0
City          0
Product       0
Quantity      0
Price         0
Total         0
Date          0
dtype: int64
Check Duplicate Records:
```

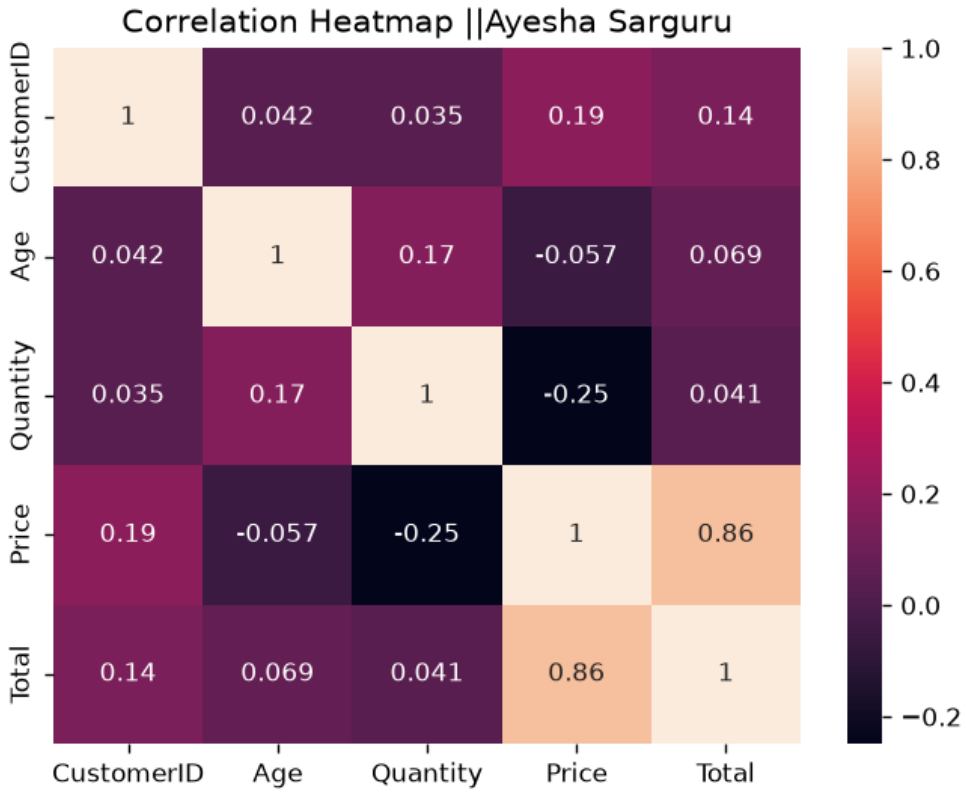



Gender Distribution || By Ayesha Sarguru
count
16
14
12
10
8
6
4
2
0
Male
Female
Gender


City-wise Customers || By Ayesha Sarguru
count
8
7
6
5
4
3
2
1
0
Mumbai
Nagpur
Thane
Pune
Nashik


Price Distribution ||Ayesha Sarguru
Frequency
Price
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
0
10000
20000
30000
40000
50000


Box Plot of Total sales ||Ayesha Sarguru
Total
0
20000
40000
60000
80000
100000
120000
140000



Business Insights / Conclusion:

The 30 sales records contain no missing values. Headphones (7 orders) are the most frequently sold product, followed by Mouse, Monitor and Keyboard (6 each) and Laptop (5). The customer base is slightly male-dominated (16 male, 14 female), and Thane (8 customers) and Mumbai (7) are the strongest cities.

The mean transaction value is ₹21,276.67, but the box plot shows two very high-value Laptop orders as outliers. Price has a strong positive correlation with Total (0.86) while Quantity has almost none, so high-priced products drive revenue and should be prioritised in inventory planning.

Practical 1B: Customer Data Analysis

Aim: To analyze customer data to measure total revenue, product-wise and city-wise sales, spending patterns and customer segments for business decision-making.

Theory: *Customer data analysis uses aggregation and grouping to convert raw transactions into business measures such as total revenue, average spending, product-wise and city-wise sales, monthly trends and customer segments. The analysis is performed on sales_dataset120Record.xlsx using Pandas.*

Pract1b.py

```
# Step 1: Import Required Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Load Dataset
df = pd.read_excel("sales_dataset120Record.xlsx")
print("Display all Records: ")
print(df)

#  Total Revenue
print("Total Revenue:")
print(df["Total"].sum())

# Average Customer Spending
print("average Customer Spending:")
df["Total"].mean()

# Product-wise Revenue
print(df.groupby("Product")["Total"].sum().sort_values(ascending=False))
# Product-wise Revenue Bar Plot
df.groupby("Product")["Total"].sum().plot(kind="bar")
plt.ylabel("Revenue")
plt.xticks(rotation=5)
plt.title("Product-wise Revenue || Ayesha Sarguru")
plt.show()


# City-wise Sales
print("City-wise Sales")
print(df.groupby("City")["Total"].sum())

# City-wise Sales Revenue Bar Plot
df.groupby("City")["Total"].sum().plot(kind="bar")
plt.title("City-wise Sales Revenue || Ayesha Sarguru")
plt.show()

# Gender-wise Spending
print("Gender-wise Spending")
print(df.groupby("Gender")["Total"].mean())

# Age Distribution
sns.histplot(df["Age"],bins=10)
plt.title("Age Distribution Histogram || Ayesha Sarguru")
plt.show()


# Monthly Sales Trend
df["Date"]=pd.to_datetime(df["Date"])

df["Month"]=df["Date"].dt.month_name()

monthly=df.groupby("Month")["Total"].sum()

monthly.plot(marker="o")
plt.title("Monthly Sales Trend || Ayesha Sarguru")
plt.show()

# Top 10 Highest Transactions
print("Top 10 Highest Transactions")
print(df.sort_values("Total",ascending=False).head(10))
```

```python
# Customer Segmentation
print("Customer Segmentation")
high=df[df["Total"]>=50000]
print("High sales customer")
print(high)

medium=df[(df["Total"]>=10000) & (df["Total"]<50000)]

print("medium sales customer")
print(medium)

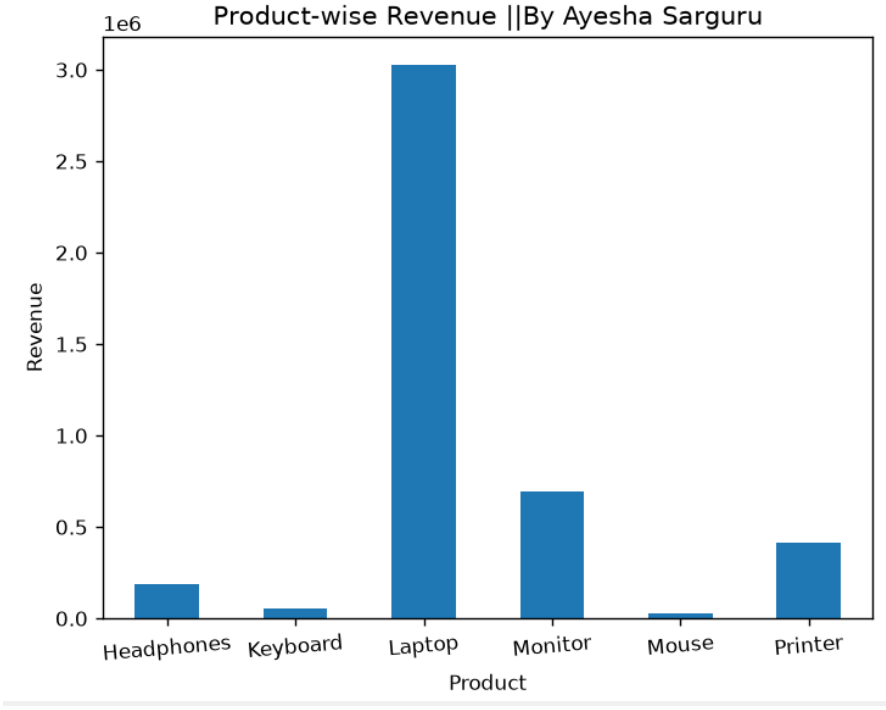

low=df[df["Total"]<10000]
print("Low sales customer")
print(low)

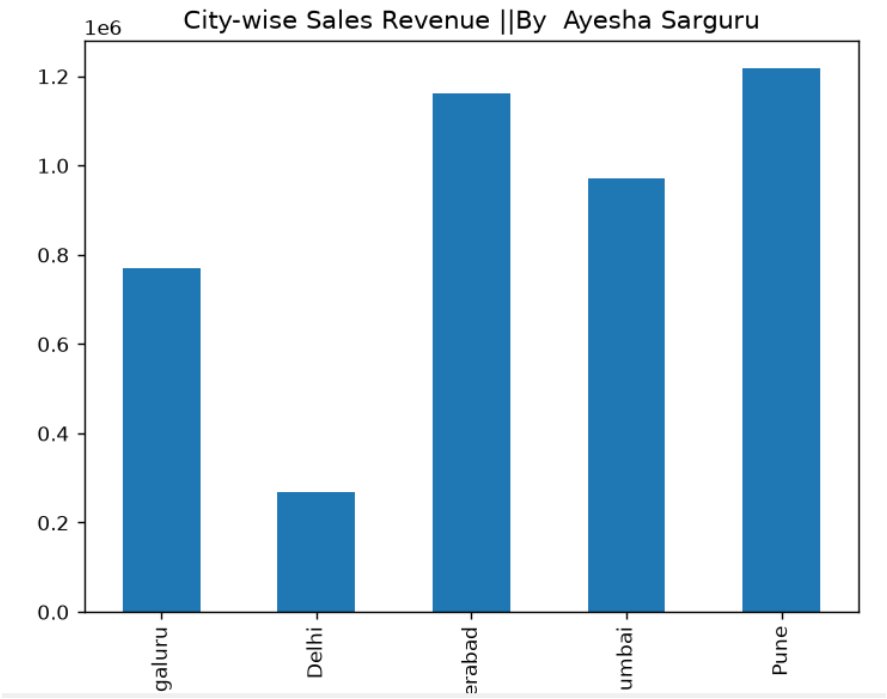
print("High Value Customers:",len(high))
print("Medium Value Customers:",len(medium))
print("Low Value Customers:",len(low))
```

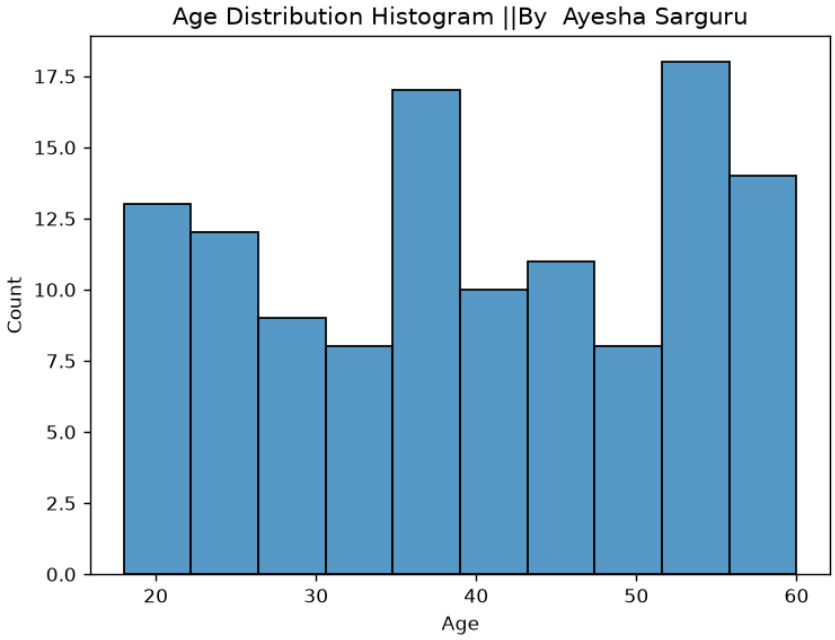
Output:

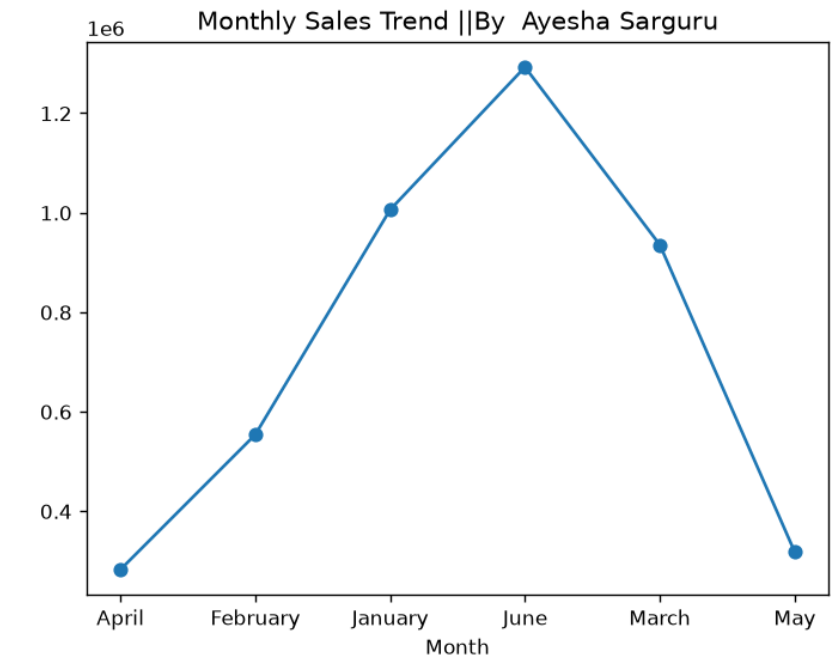
```
[Running] python -u "d:\business analytics\Prac1b.py"
Display all Records:
     CustomerID  Gender  Age       City  ... Quantity  Price   Total        Date
0          1001  Female   33       Pune  ...        1   2262    2262  2026-02-05
1          1002    Male   55  Bengaluru  ...        1   2588    2588  2026-01-09
2          1003    Male   53       Pune  ...        1  47186   47186  2026-06-16
3          1004  Female   18       Pune  ...        4   2360    9440  2026-06-28
4          1005    Male   39     Mumbai  ...        3  11466   34398  2026-01-24
..          ...     ...  ...        ...  ...      ...    ...     ...         ...
115        1116    Male   53     Mumbai  ...        1   2328    2328  2026-02-10
116        1117  Female   36     Mumbai  ...        4  49505  198020  2026-03-01
117        1118  Female   22       Pune  ...        3   1560    4680  2026-03-09
118        1119    Male   59       Pune  ...        2   2462    4924  2026-03-10
119        1120  Female   56  Hyderabad  ...        1    638     638  2026-03-15

[120 rows x 9 columns]
Total Revenue:
4387429
average Customer Spending:
Product
Laptop        3025121
Monitor        692028
Printer        409917
Headphones     185441
Keyboard        51348
Mouse           23574
Name: Total, dtype: int64
```




City-wise Sales Revenue ||By Ayesha Sarguru
1e6
1.2
1.0
0.8
0.6
0.4
0.2
0.0
galuru
Delhi
umbai
Pune


Age Distribution Histogram ||By Ayesha Sarguru
Count
17.5
15.0
12.5
10.0
7.5
5.0
2.5
0.0
20
30
40
50
60
Age



Business Insights / Conclusion:

The business earned a total revenue of ₹43,87,429 from 120 transactions, an average of about ₹36,562 per transaction. Laptops generate ₹30,25,121 (about 69% of revenue), followed by Monitors (₹6,92,028) and Printers (₹4,09,917), while Mice and Keyboards contribute very little.

Pune (₹12.18 lakh) and Hyderabad (₹11.60 lakh) are the top cities and Delhi is the weakest (₹2.69 lakh); male and female customers spend almost equally. Sales peak in June (₹12.92 lakh) and are lowest in April (₹2.82 lakh). With 25 high-value, 37 medium-value and 58 low-value customers, loyalty programmes should target the high-value segment.

Practical 2A: Descriptive Statistics

Aim: To calculate descriptive statistics (mean, median, mode, variance, standard deviation and range) for a business variable of interest.

Theory: *Descriptive statistics summarize a variable using measures of central tendency (mean, median, mode) and dispersion (variance, standard deviation, range). These measures describe typical customer spending in the Total column of sales.csv.*

Pract2a.py

```
'''Pract 2. Describing the Distribution of a Variable:

a. Obtain a dataset and calculate descriptive statistics (mean, median, mode,
variance, etc.) for a specific variable of interest.'''



# Variables of Interest
# We will analyze the Total column.

# Step 1: Import Required Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Load the Dataset
df = pd.read csv("sales.csv")

# Step 3: Display First Five Records
print("Display First Five Records")
print(df.head())

# Step 4: Select the Variable
sales = df["Total"]
print("Selected Variable : Total - Sales")
print(sales)
# Calculate Descriptive Statistics
# Step 5: Display Summary Statistics
print("Display Summary Statistics")
print(sales.describe())

# Step 6: Calculate Mean
print("Mean:")
print(sales.mean())

# Step 7: Calculate Median
print("Median")
print(sales.median())

# Step 8: Calculate Mode
print("Mode:")
print(sales.mode())

# Step 9: Calculate Variance
print("Variance:")
print(sales.var())

# Step 10: Calculate Standard Deviation
print("Standard Deviation:")
print(sales.std())

# Step 11: Calculate Minimum and Maximum
print("Minimum Sales :", sales.min())
print("Maximum Sales :", sales.max())

# Step 12: Calculate Range
print("Range:")
print(sales.max() - sales.min())
```

Output:

```
Display First Five Records
   CustomerID  Gender  Age    City   Product  Quantity  Price  Total        Date
0         101    Male   28  Mumbai     Mouse         3    600   1800  02-01-2026
1         102  Female   40  Nagpur   Monitor         2  15000  30000  03-01-2026
2         103  Female   20  Nagpur     Mouse         1    600    600  04-01-2026
3         104    Male   42  Nagpur   Monitor         3  15000  45000  05-01-2026
4         105    Male   38  Mumbai  Keyboard         3   1200   3600  06-01-2026
Selected Variable : Total - Sales
0       1800
1      30000
2        600
3      45000
4       3600
5       1200
6     100000
7       2500
8       2500
9      30000
10     50000
11      5000
12      5000
13     15000
14      3600
15      2500
16      2400
17      3600
18      1800
19      7500
20      3600
21      7500
22      1800
23     30000
24     30000
25    150000
26     50000
27       600
28     50000
29      1200
```

```
28     50000
29      1200
Name: Total, dtype: int64
Display Summary Statistics
count        30.000000
mean      21276.666667
std       33487.657874
min         600.000000
25%        2425.000000
50%        4300.000000
75%       30000.000000
max      150000.000000
Name: Total, dtype: float64
Mean:
21276.666666666668
Median
4300.0
Mode:
0     3600
1    30000
Name: Total, dtype: int64
Variance:
1121423229.8850574
Standard Deviation:
33487.65787398482
Minimum Sales : 600
Maximum Sales : 150000
Range:
149400
By  Ayesha Sarguru
PS D:\business analytics>
```

Business Insights / Conclusion:

The mean transaction value is ₹21,276.67, far higher than the median of ₹4,300, which shows that a few very large purchases pull the average upward. The data has two modes (₹3,600 and ₹30,000), and the standard deviation of ₹33,487.66 is larger than the mean.

Sales range from ₹600 to ₹1,50,000 (range ₹1,49,400), indicating very uneven customer spending. The median is therefore a better indicator of a typical purchase than the mean.

Practical 2B: Distribution Visualization

Aim: To visualize the distribution of a business variable using histograms, box plots and violin plots and to identify outliers.

Theory: *Histograms, box plots and violin plots show the shape, spread and skewness of a variable, while the IQR rule (Q1 − 1.5×IQR, Q3 + 1.5×IQR) flags unusually high or low values as outliers. The Total column of sales.csv is visualized.*

Pract2b.py

```
# Step 1: Import Required Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Load the Dataset
df = pd.read_csv("sales.csv")

# Step 3: Display First Five Records
print("Display First Five Records")
print(df.head())

# Step 4: Select the Variable
sales = df["Total"]
print("Selected Variable : Total - Sales")
print(sales)
# Visualize the Distribution
# Step 5: Histogram
plt.figure(figsize=(8,5))
plt.hist(sales, bins=10, edgecolor="black")
plt.title("Distribution of Total Sales")
plt.xlabel("Total Sales")
plt.ylabel("Frequency")
plt.show()

# Step 6: Histogram with KDE Curve
plt.figure(figsize=(8,5))
sns.histplot(sales, bins=10, kde=True)
plt.title("Histogram of Total Sales")
plt.show()

# The KDE (Kernel Density Estimate) curve provides a smoother view of the data distribution.

# Step 7: Box Plot
plt.figure(figsize=(6,4))
sns.boxplot(y=sales)
plt.title("Box Plot of Total Sales")
plt.show()

# Step 8: Violin Plot (Optional)
plt.figure(figsize=(6,4))
sns.violinplot(y=sales)
plt.title("Violin Plot of Total Sales")
plt.show()

# Step 9: Identify Outliers
Q1 = sales.quantile(0.25)
Q3 = sales.quantile(0.75)

IQR = Q3 - Q1

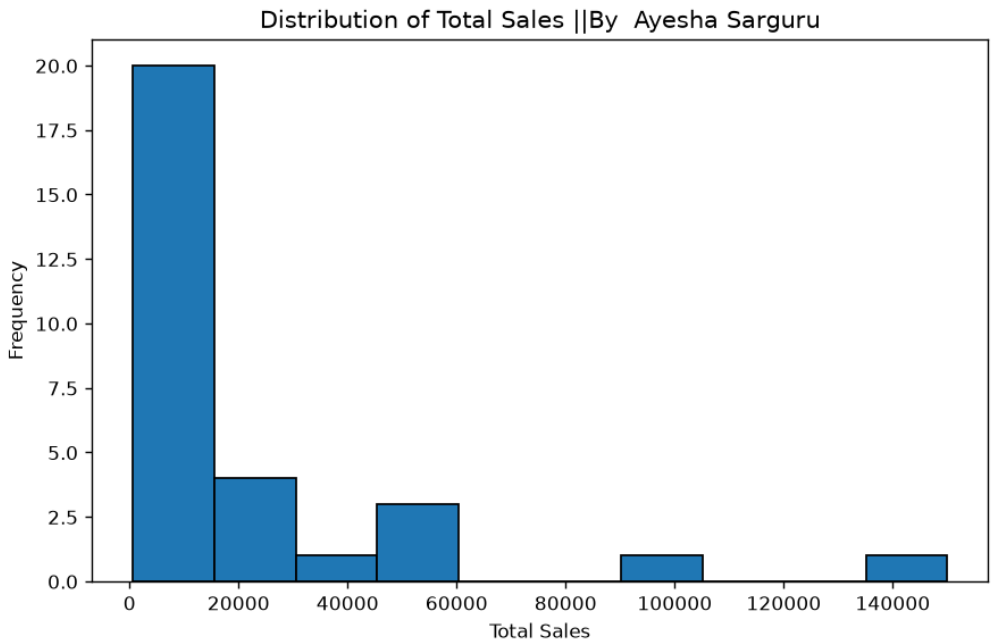
Lower = Q1 - 1.5 * IQR
Upper = Q3 + 1.5 * IQR
print( "Lower outlier value:",Lower)

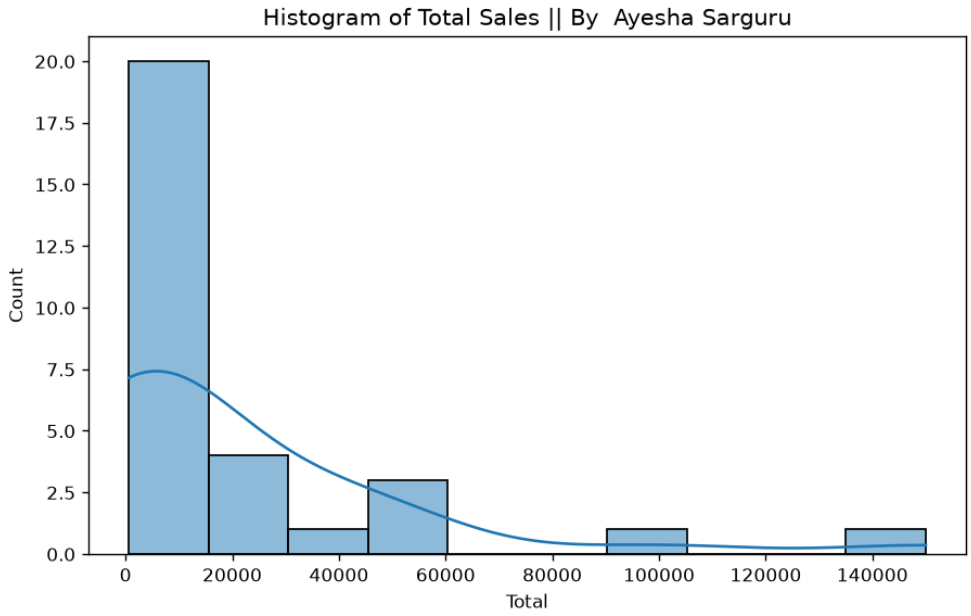
print( "Upper outlier value:",Upper)
outliers = df[(sales < Lower) | (sales > Upper)]
print("outliers")
print(outliers)

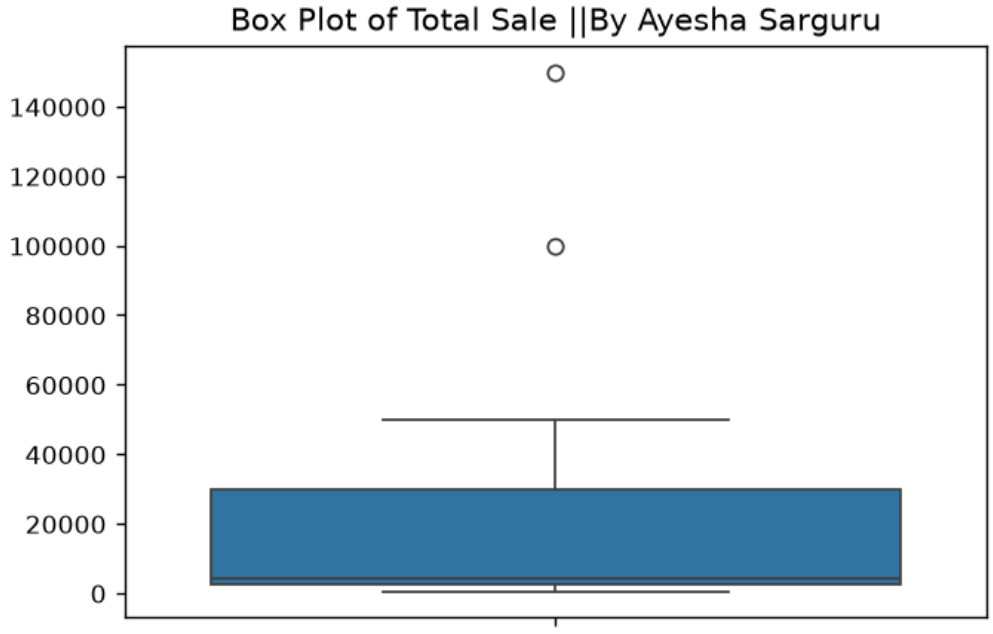
# Outliers represent unusually high or low transactions that may require further
investigation.
```

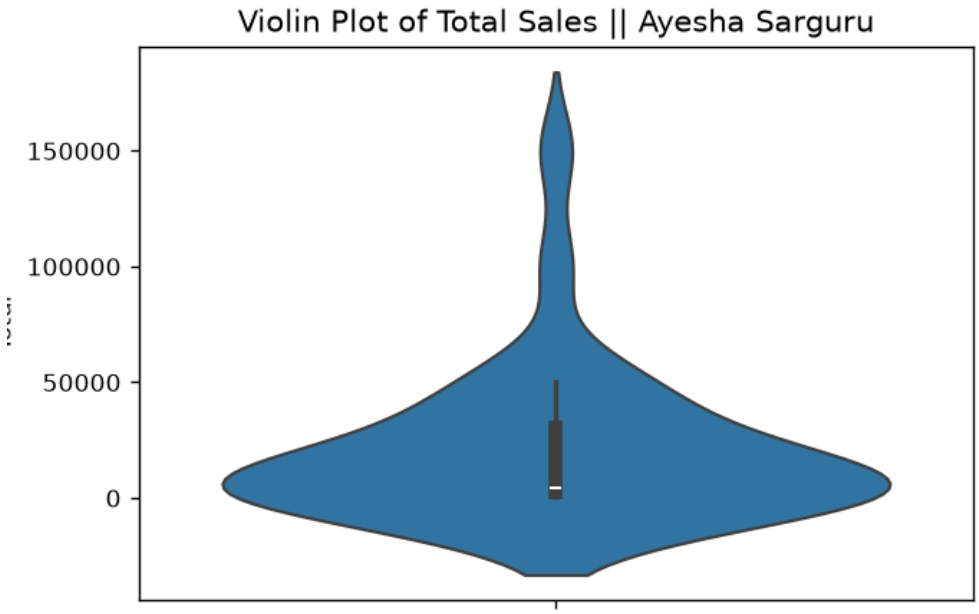
Output:

```
PS D:\business analytics> & C:\Users\Student\AppData\Local\Microsoft\WindowsApps\python
By  Ayesha Sarguru
Display First Five Records
   CustomerID  Gender  Age    City   Product  Quantity  Price  Total        Date
0         101    Male   28  Mumbai     Mouse         3    600   1800  02-01-2026
1         102  Female   40  Nagpur   Monitor         2  15000  30000  03-01-2026
2         103  Female   20  Nagpur     Mouse         1    600    600  04-01-2026
3         104    Male   42  Nagpur   Monitor         3  15000  45000  05-01-2026
4         105    Male   38  Mumbai  Keyboard         3   1200   3600  06-01-2026
Selected Variable : Total - Sales
0       1800
1      30000
2        600
3      45000
4       3600
5       1200
6     100000
7       2500
8       2500
9      30000
10     50000
11      5000
12      5000
13     15000
14      3600
15      2500
16      2400
17      3600
18      1800
19      7500
20      3600
21      7500
22      1800
23     30000
24     30000
25    150000
26     50000
27       600
28     50000
29      1200
Name: Total, dtype: int64
Lower outlier value: -38937.5
Upper outlier value: 71362.5
outliers
    CustomerID  Gender  Age    City Product  Quantity  Price   Total        Date
6          107    Male   33  Mumbai  Laptop         2  50000  100000  08-01-2026
25         126  Female   34   Thane  Laptop         3  50000  150000  27-01-2026
PS D:\business analytics>
```




Histogram of Total Sales || By Ayesha Sarguru
Count
Total
0.0
2.5
5.0
7.5
10.0
12.5
15.0
17.5
20.0
0
20000
40000
60000
80000
100000
120000
140000


Box Plot of Total Sale ||By Ayesha Sarguru
0
20000
40000
60000
80000
100000
120000
140000



Business Insights / Conclusion:

The histogram and KDE curve are strongly right-skewed: most transactions are below ₹20,000, while a few reach ₹1,00,000 and ₹1,50,000. Using the IQR rule (lower limit −₹38,937.5, upper limit ₹71,362.5), the two Laptop purchases of ₹1,00,000 and ₹1,50,000 are outliers.

These probably represent premium or bulk orders and should be studied separately for pricing, targeted offers and bulk-sale opportunities.

Practical 3A: Correlation Analysis

Aim: To perform correlation analysis to determine the strength and direction of relationships between advertising channels and sales revenue.

Theory: *Correlation measures the strength and direction of the linear relationship between two variables on a scale from −1 to +1. A correlation matrix and heatmap compare TV, Radio and Newspaper advertising with sales in advertising_sales Pract3.csv.*

Pract3a.py

```
'''Finding Relationships Among Variables:
3.a. Use a dataset with multiple variables
and perform correlation analysis to
determine the strength and direction of relationships between
pairs of variables. '''

# Step 1: Import Required Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns

# Step 2: Load Dataset
df = pd.read_csv("advertising_sales Pract3.csv")


# Step 3: Display Dataset
print("Display Dataset : head")
print(df.head())


# Step 4: Check Dataset Information
print("Check Dataset Information")
print(df.info())


# Step 5: Calculate Correlation Matrix
correlation = df[['TV Advertising',
                  'Radio Advertising',
                  'Newspaper_Advertising',
                  'Sales_Revenue']].corr()

print(correlation.to_string())


# Step 6: Display Correlation Heatmap
plt.figure(figsize=(8,6))

sns.heatmap(correlation,
            annot=True,
            cmap="coolwarm",
            linewidths=0.5)

plt.title("Correlation Matrix")

plt.show()

# Step 7: Scatter Plot (TV Advertising vs Sales Revenue)
plt.figure(figsize=(7,5))

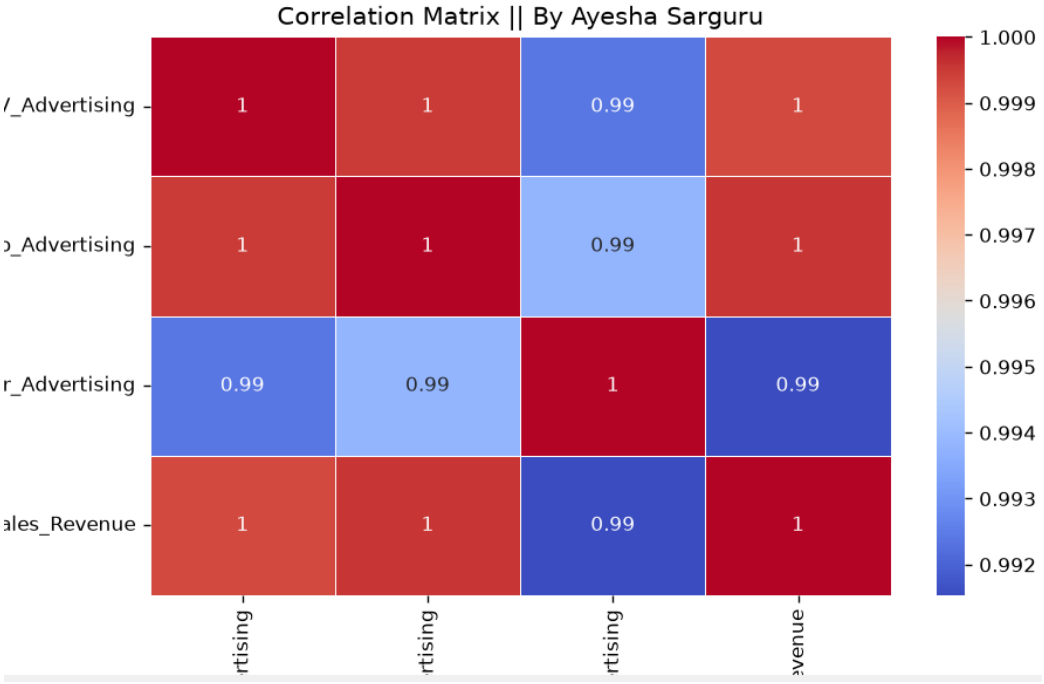
sns.scatterplot(data=df,
                x="TV_Advertising",
                y="Sales_Revenue")

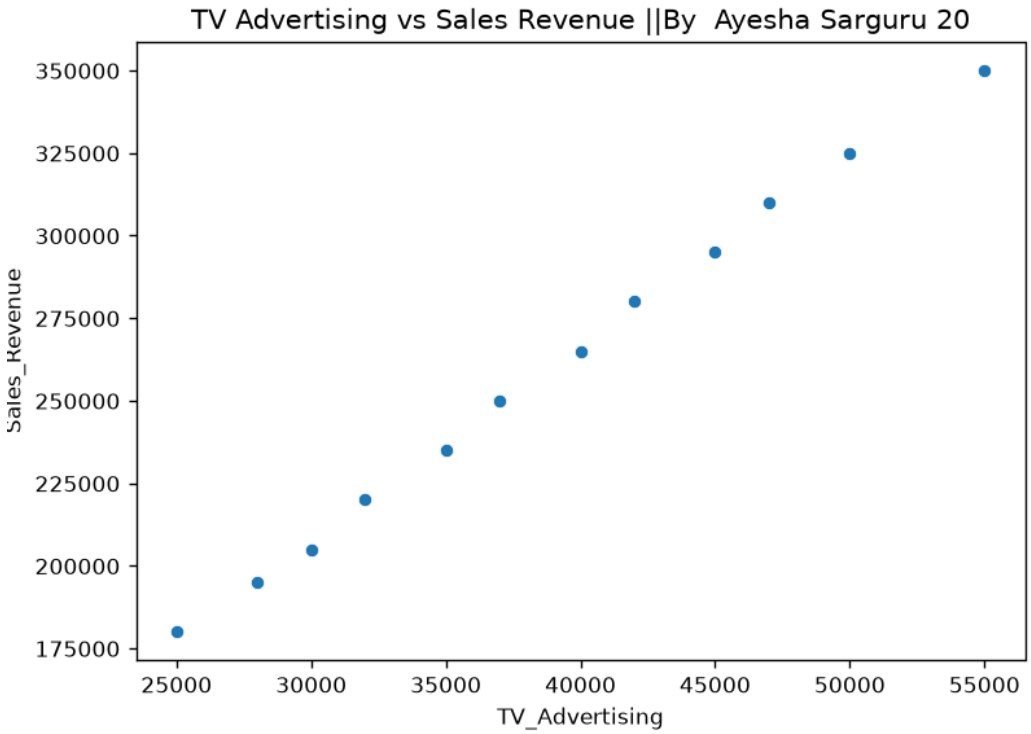
plt.title("TV Advertising vs Sales Revenue")

plt.show()
```

Output:

```
PS D:\business analytics> & C:\Users\Student\AppData\Local\Microsoft\WindowsApps\python3.12.exe
By Ayesha Sarguru 20
Display Dataset : head
   Month  TV_Advertising  Radio_Advertising  Newspaper_Advertising  Sales_Revenue
0    Jan           25000               8000                   3000         180000
1    Feb           28000               8500                   3500         195000
2    Mar           30000               9000                   4000         205000
3    Apr           32000               9500                   4200         220000
4    May           35000              10000                   4500         235000
Check Dataset Information
<class 'pandas.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 5 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Month                 12 non-null     str
 1   TV_Advertising        12 non-null     int64
 2   Radio_Advertising     12 non-null     int64
 3   Newspaper_Advertising  12 non-null     int64
 4   Sales_Revenue         12 non-null     int64
dtypes: int64(4), str(1)
memory usage: 612.0 bytes
None
                      TV_Advertising  Radio_Advertising  Newspaper_Advertising  Sales_Revenue
TV_Advertising              1.000000           0.999476               0.992387       0.999297
Radio_Advertising           0.999476           1.000000               0.993809       0.999560
Newspaper_Advertising       0.992387           0.993809               1.000000       0.991532
Sales_Revenue               0.999297           0.999560               0.991532       1.000000
PS D:\business analytics>
```





Business Insights / Conclusion:

All three advertising channels are strongly and positively correlated with sales revenue: Radio (0.9996), TV (0.9993) and Newspaper (0.9915), with Newspaper the weakest. The differences between the channels are very small.

Because all three budgets increase together month after month, correlation alone cannot show which channel causes the sales growth, but it confirms that higher advertising spend is associated with higher revenue.

Practical 3B: Regression Analysis

Aim: To apply linear regression analysis to identify the relationship between TV advertising expenditure (independent variable) and sales revenue (dependent variable).

Theory: *Simple linear regression fits the line Y = b0 + b1X, where the slope shows the change in sales for every ₹1 spent on TV advertising and R² shows how much variation the model explains. Scikit-learn's LinearRegression is trained on advertising_sales Pract3.csv.*

Pract3b.py

```
'''3.b Apply regression analysis to identify the relationship between an independent
variable (e.g., advertising expenditure) and a dependent variable (e.g., sales
revenue). '''

# Step 1: Import Required Libraries
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.linear_model import LinearRegression

# Step 2: Load Dataset
df = pd.read csv("advertising sales Pract3.csv")


# Step 3: Display Dataset
print("Display Dataset : head")
print(df.head())


# Step 4: Check Dataset Information
print("Check Dataset Information")
print(df.info())

# Step 2: Define Independent and Dependent Variables
X = df[['TV_Advertising']]

y = df['Sales Revenue']

# Step 3: Train the Regression Model
model = LinearRegression()

model.fit(X, y)

# Step 4: Display Regression Equation
print("Intercept :", model.intercept_)

print("Slope :", model.coef_[0])
'''Interpretation
•      Intercept: Estimated sales revenue when TV advertising expenditure is zero.
•      Slope: Increase in sales revenue for every additional ₹1 spent on TV advertising. '''


# Step 5: Predict Sales Revenue
predicted sales = model.predict(X)

print(predicted_sales)

# Step 6: Calculate R² Score
r2 = model.score(X, y)

print("R² Score :", r2)
'''Interpretation
R² close to 1 indicates that TV advertising explains most of the variation in sales revenue.
R² close to 0 indicates a weak predictive model. '''


# Step 7: Plot Regression Line
plt.figure(figsize=(8,5))

plt.scatter(X, y, color='blue')

plt.plot(X,
         predicted_sales,
         color='red',
```

```
          linewidth=2)

plt.xlabel("TV Advertising (₹)")

plt.ylabel("Sales Revenue (₹)")

plt.title("Linear Regression: TV Advertising vs Sales Revenue")

plt.show()

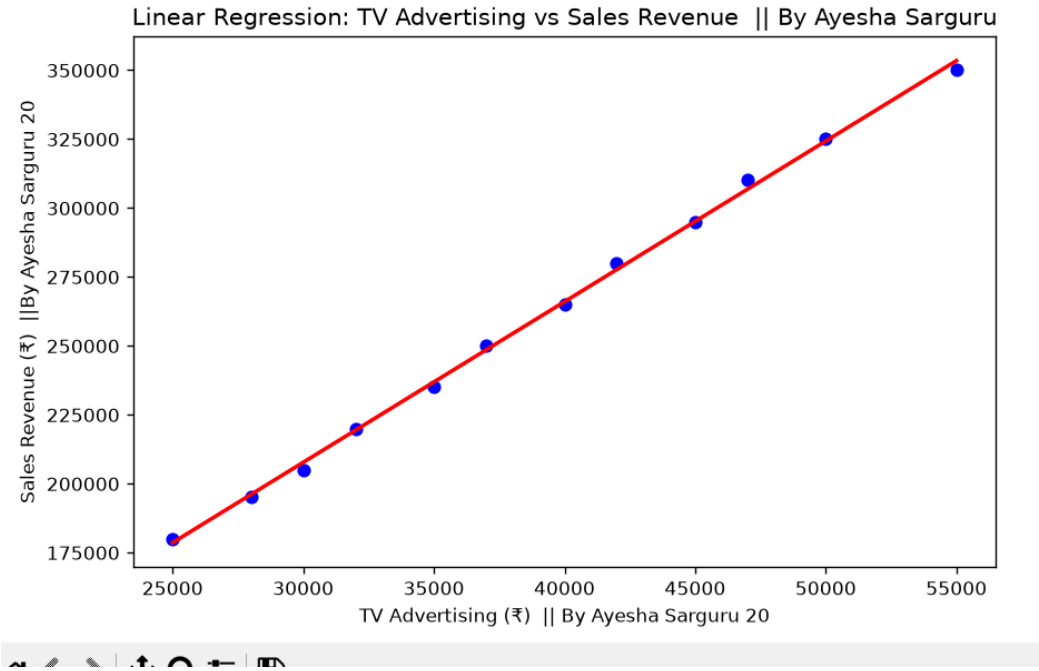
# Step 8: Predict Future Sales
# Predict sales if the company spends ₹50,000 on TV advertising.
future = [[50000]]

prediction = model.predict(future)

print("Predicted Sales Revenue =", prediction)
```

Output:

```
PS D:\business analytics> & C:\Users\Student\AppData\Local\Microsoft\WindowsApps\py
By Ayesha Sarguru 20
Display Dataset : head
   Month  TV_Advertising  Radio_Advertising  Newspaper_Advertising  Sales_Revenue
0   Jan           25000               8000                   3000         180000
1   Feb           28000               8500                   3500         195000
2   Mar           30000               9000                   4000         205000
3   Apr           32000               9500                   4200         220000
4   May           35000              10000                   4500         235000
Check Dataset Information
<class 'pandas.DataFrame'>
RangeIndex: 12 entries, 0 to 11
Data columns (total 5 columns):
 #   Column                Non-Null Count  Dtype
---  ------                --------------  -----
 0   Month                 12 non-null     str
 1   TV_Advertising        12 non-null     int64
 2   Radio_Advertising     12 non-null     int64
 3   Newspaper_Advertising  12 non-null     int64
 4   Sales_Revenue         12 non-null     int64
dtypes: int64(4), str(1)
memory usage: 612.0 bytes
None
Intercept : 32831.17790982171
Slope : 5.8283816847256205
[178540.72002796 196025.86508214 207682.62845159 219339.39182104
 236824.53687522 248481.30024467 265966.44529885 277623.2086683
 295108.35372247 306765.11709193 324250.2621461  353392.17056973]
R² Score : 0.9985950638869737
C:\Users\Student\AppData\Local\Packages\PythonSoftwareFoundation.Python.3.12_qbz5n2
rWarning: X does not have valid feature names, but LinearRegression was fitted with
  warnings.warn(
Predicted Sales Revenue = [324250.2621461]
PS D:\business analytics>
```



Business Insights / Conclusion:

The regression equation is Sales = 32,831.18 + 5.83 × TV Advertising: every additional ₹1 spent on TV advertising is associated with about ₹5.83 of extra sales revenue. The R^2 score of 0.9986 shows that TV spend explains almost all of the variation in sales.

The model predicts a sales revenue of about ₹3,24,250 for a TV budget of ₹50,000 and can support advertising budget planning within the range of the observed data.

Practical 4: Business Intelligence Tools (Tableau)

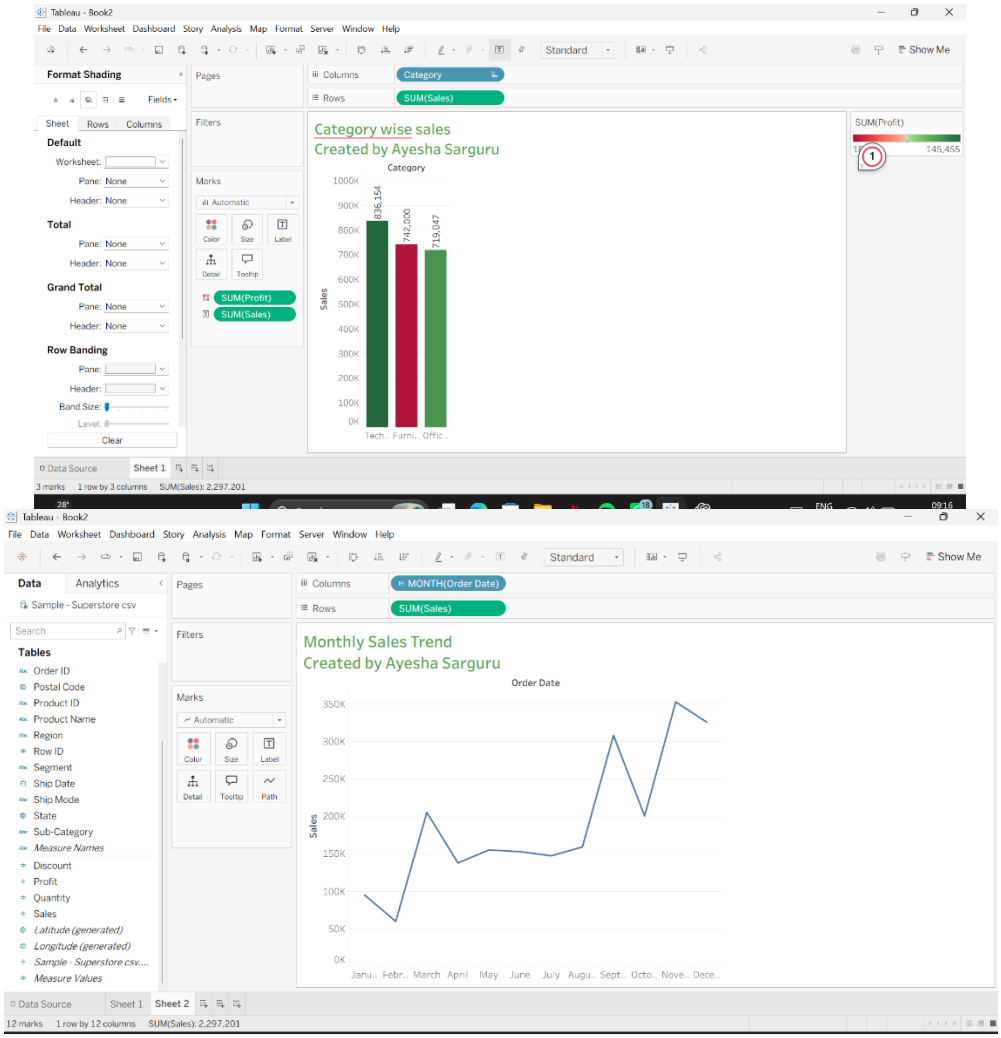
Aim: To utilize a Business Intelligence (BI) tool such as Tableau to extract meaningful insights from a dataset and create interactive visualizations for effective business decision-making.

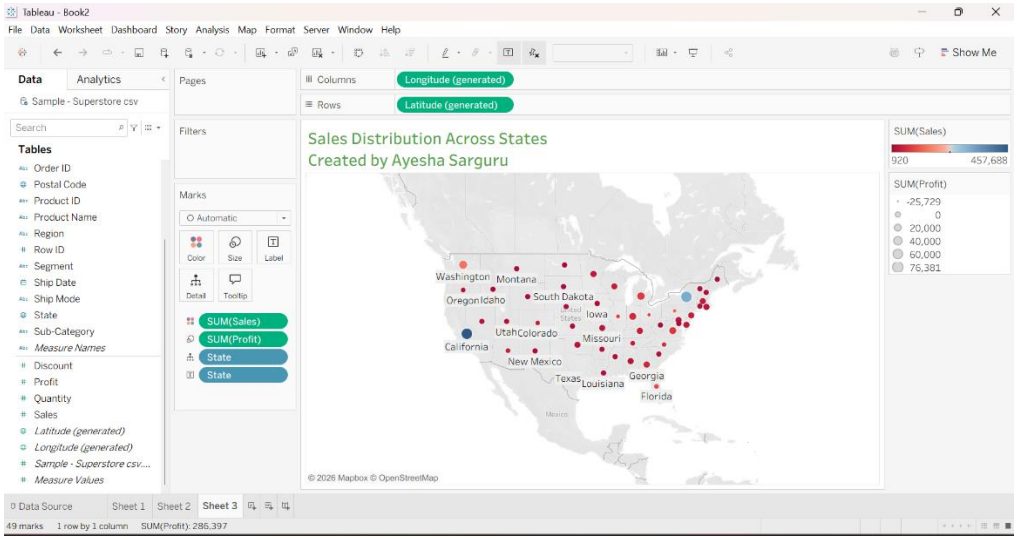
Theory: *Business Intelligence (BI) tools such as Tableau transform raw data into interactive visualizations and dashboards. The Sample Superstore dataset is connected to Tableau and analyzed using KPI cards, charts, maps, and filters to identify sales trends, profitability, and regional performance.*

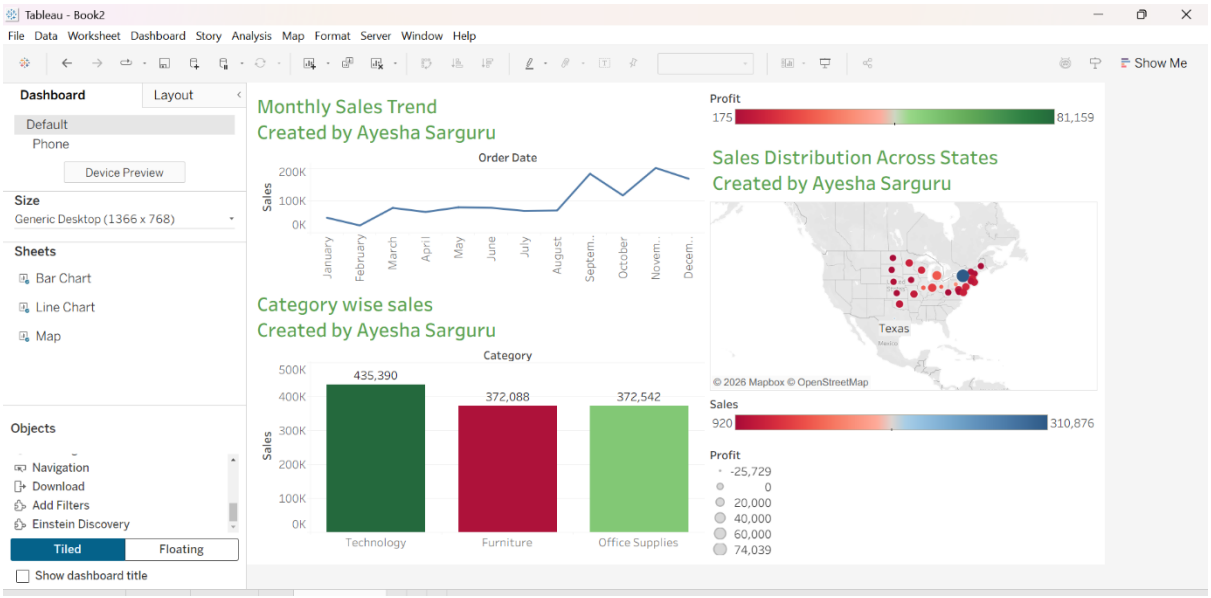
Tableau Calculated Fields

1. **Total Sales:** SUM([Sales])
2. **Total Profit:** SUM([Profit])
3. **Total Quantity Sold:** SUM([Quantity])
4. **Profit Ratio:** SUM([Profit]) / SUM([Sales])
5. **Average Discount:** AVG([Discount])
6. **Total Orders:** COUNTD([Order ID])

Output:







Business Insights / Conclusion:

The Tableau dashboard was created using the Sample Superstore dataset. It includes a monthly sales trend line chart, a category-wise sales bar chart, and a geographical map showing sales distribution across states. Profit is represented using color and size indicators to compare business performance across different regions. The dashboard helps users analyze sales, profit, and category performance effectively.

Practical 5A: Probability Simulation

Aim: To simulate a probability experiment, such as rolling a die, using Python and calculate the probabilities of different outcomes.

Theory: *Probability is the likelihood of an event, P(E) = favourable outcomes / total outcomes. For a fair die the theoretical probability of each face is 1/6 = 0.1667, and the experimental probability from 10,000 simulated rolls approaches this value as the number of trials increases.*

Pract5a.py

```
import random
from collections import Counter

# Number of times the die is rolled
n = 10000

# Simulate rolling a fair die
rolls = [random.randint(1, 6) for _ in range(n)]
print(rolls)
# Count occurrences of each outcome
counts = Counter(rolls)
print(counts)
print("Outcome\tFrequency\tExperimental Probability")

for outcome in range(1, 7):
    frequency = counts[outcome]
    probability = frequency / n
    print(outcome, "\t", frequency, "\t\t", round(probability, 4))

# Theoretical probability
print("\nTheoretical Probability of each outcome =", round(1/6, 4))
```

Output:

```
PS D:\business analytics> & C:\Users\Student\AppData\Local\Microsoft\WindowsApps\python3.12.exe "d:/business analytics/PRAC5A.PY"
Counter({5: 1730, 4: 1689, 3: 1688, 2: 1657, 6: 1647, 1: 1589})
Outcome Frequency       Experimental Probability
1       1589            0.1589
2       1657            0.1657
3       1688            0.1688
4       1689            0.1689
5       1730            0.173
6       1647            0.1647

Theoretical Probability of each outcome = 0.1667
By  Ayesha Sarguru 20
PS D:\business analytics>
```

Business Insights / Conclusion:

After 10,000 rolls the experimental probabilities of the six faces range from 0.1589 to 0.1730 and are close to the theoretical value of 0.1667. The small differences are due to randomness and shrink as the number of trials grows.

Such simulations help businesses model uncertain events like customer purchases, demand and risk.

Practical 5B: Probability Distributions

Aim: To generate random numbers from Normal, Uniform, and Exponential probability distributions using Python and analyze their properties.

Theory: *A probability distribution describes how the values of a random variable are spread. NumPy generates random observations from the Normal (bell-shaped, mean μ and SD σ), Uniform (equally likely values) and Exponential (right-skewed, waiting-time) distributions, and Matplotlib histograms visualize them.*

Pract5b.py

```
import numpy as np
import matplotlib.pyplot as plt

# Number of random observations
n = 10000

# Generate random numbers

# Normal distribution
normal data = np.random.normal(
    loc=50,
    scale=10,
    size=n
)

# Uniform distribution
uniform_data = np.random.uniform(
    low=0,
    high=100,
    size=n
)

# Exponential distribution
exponential_data = np.random.exponential(
    scale=10,
    size=n
)


# Display first 10 values
print("Normal Distribution:")
print(normal_data[:100])

print("\nUniform Distribution:")
print(uniform_data[:100])

print("\nExponential Distribution:")
print(exponential data[:100])

# Calculate properties
print("\nStatistical Properties")

print("\nNormal Distribution")
print("Mean:", np.mean(normal data))
print("Standard Deviation:", np.std(normal_data))

print("\nUniform Distribution")
print("Mean:", np.mean(uniform_data))
print("Standard Deviation:", np.std(uniform_data))

print("\nExponential Distribution")
print("Mean:", np.mean(exponential_data))
print("Standard Deviation:", np.std(exponential_data))

# Visualizing the Distributions

# Normal Distribution
plt.figure(figsize=(8, 5))
plt.hist(normal data, bins=30)
plt.title("Normal Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
```

```
plt.show()
# Uniform Distribution
plt.figure(figsize=(8, 5))
plt.hist(uniform_data, bins=30)
plt.title("Uniform Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
# Exponential Distribution
plt.figure(figsize=(8, 5))
plt.hist(exponential_data, bins=30)
plt.title("Exponential Distribution")
plt.xlabel("Value")
plt.ylabel("Frequency")
plt.show()
```

Output:

```
PS D:\business analytics> & C:\Users\Student\AppData\Local\Microsoft\WindowsApps\p
Ayesha Sarguru 20
Normal Distribution:
[46.76603051 62.88010454 49.60395647 61.21782077 30.58247978 27.82269249
 50.75257726 33.64103687 74.59648179 42.14368802]

Uniform Distribution:
[57.82526922 91.02825612 71.09851507 40.02119418 96.10996774 80.61460343
 86.27223263 82.08856459 68.36761703 25.91984296]

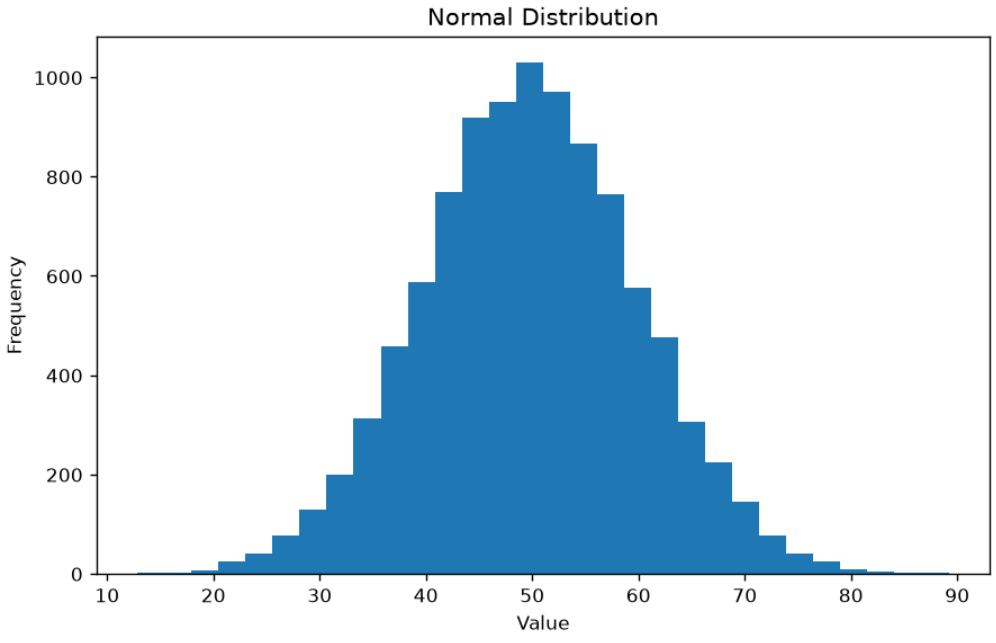
Exponential Distribution:
[10.0060408   4.27678166 10.91501971  2.74757855  2.41534129  0.09576485
 10.57020124  1.74373442  4.16612168  0.90229353]

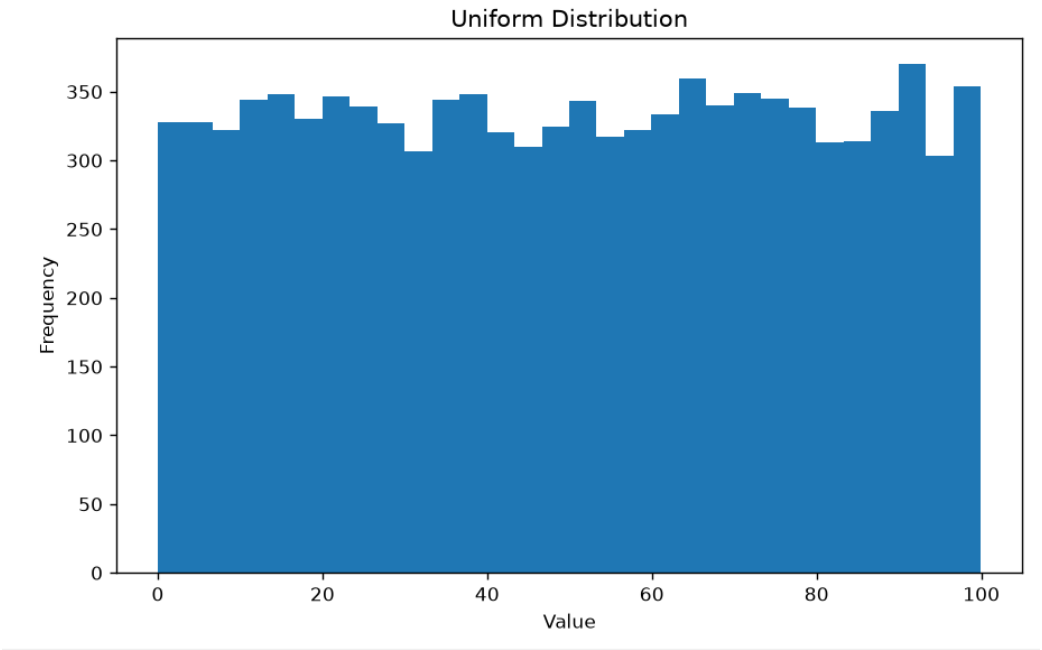
Statistical Properties

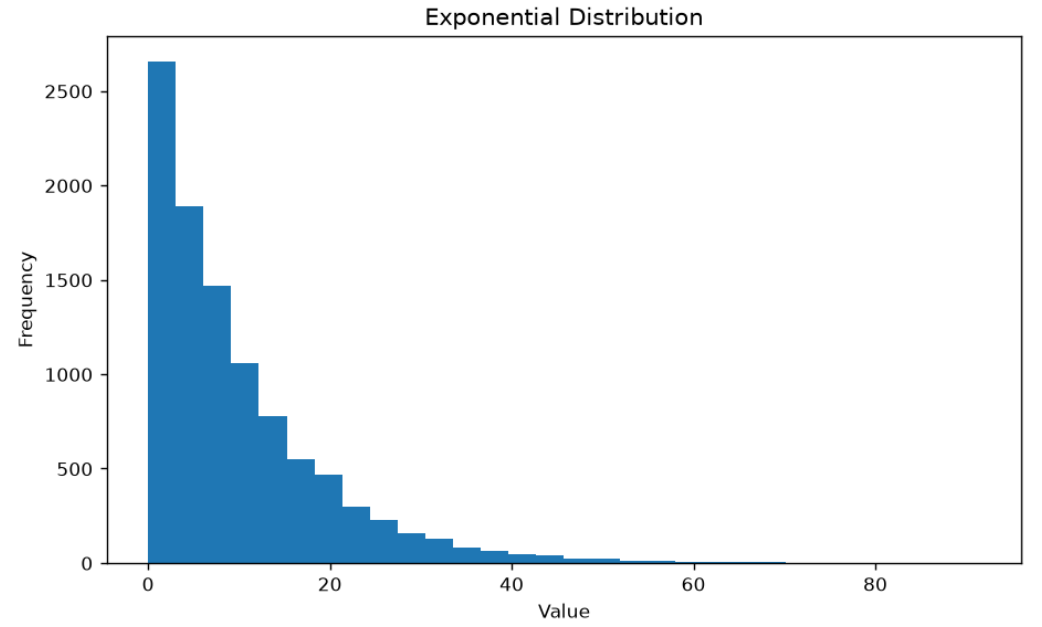
Normal Distribution
Mean: 49.81581923070171
Standard Deviation: 10.04380177171037

Uniform Distribution
Mean: 50.09896299528344
Standard Deviation: 28.891415088175144

Exponential Distribution
Mean: 9.909725896817402
Standard Deviation: 9.816587277236378
PS D:\business analytics>
```







Business Insights / Conclusion:

The Normal data has a mean of 49.82 and a standard deviation of 10.04, close to the specified $\mu = 50$ and $\sigma = 10$, with a bell-shaped histogram. The Uniform data has a mean of 50.10 and SD of 28.89 with a flat histogram, and the Exponential data has a mean of 9.91 and SD of 9.82 with a right-skewed histogram.

These distributions can model measurements and scores (Normal), random events (Uniform) and waiting times between customer arrivals (Exponential).

Practical 6: Decision Making under Uncertainty

Aim: To develop a decision tree model for business decisions under uncertainty and apply Expected Monetary Value (EMV) to select the optimal alternative.

Theory: *A decision tree models alternatives, uncertain events and payoffs. Expected Monetary Value (EMV) is the probability-weighted average payoff, EMV = Σ(probability × payoff), and the alternative with the highest EMV is the optimal decision.*

pract6.py

```
# Decision Tree: Product Selection

# Product A
p_high_A = 0.60
p_low_A = 0.40

profit_high_A = 80000
profit_low_A = -20000

# Product B
p high B = 0.70
p low B = 0.30

profit_high_B = 60000
profit low B = 10000

# Display decision tree information

print("DECISION TREE")
print("=======================")

print("\nProduct A")
print("  High Demand:", p high A, "-> Rs", profit high A)
print("  Low Demand :", p low A, "-> Rs", profit low A)

print("\nProduct B")
print("  High Demand:", p high B, "-> Rs", profit high B)
print("  Low Demand :", p low B, "-> Rs", profit low B)


# Expected Monetary Value Calculation

# Product A
ev_A = (p_high_A * profit_high_A) + \
       (p_low_A * profit_low_A)

# Product B
ev_B = (p_high_B * profit_high_B) + \
       (p low B * profit low B)

print("\nEXPECTED MONETARY VALUE")
print("=======================")

print("Expected Value of Product A: Rs", ev A)
print("Expected Value of Product B: Rs", ev B)

# Select the optimal decision
if ev_A > ev_B:
    print("\nOptimal Decision: Select Product A")
elif ev_B > ev_A:
    print("\nOptimal Decision: Select Product B")
else:
    print("\nBoth alternatives have the same Expected Value")


import matplotlib.pyplot as plt

products = ["Product A", "Product B"]
expected_values = [ev_A, ev_B]

plt.figure(figsize=(7, 5))
plt.bar(products, expected_values)

plt.title("Expected Monetary Value Comparison")
```

```
plt.xlabel("Business Alternatives")
plt.ylabel("Expected Value (Rs)")

plt.show()
```

Output:

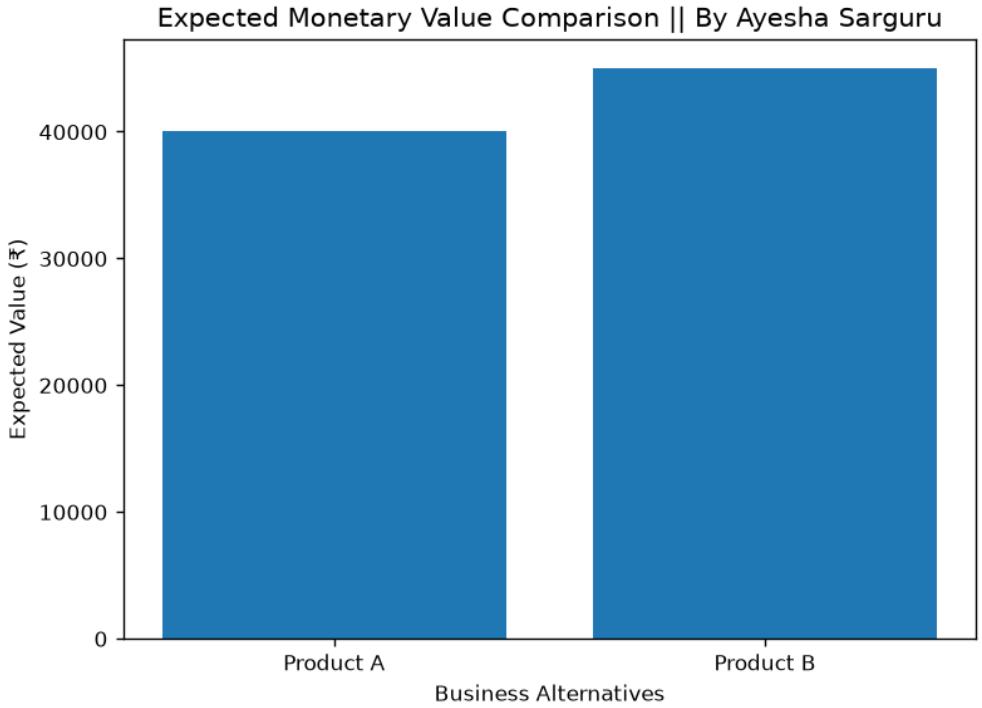
```
PS D:\business analytics> & C:\Users\Student\AppData\Local\Microsoft\Wi
By Ayesha Sarguru 20
DECISION TREE
========================

Product A
  High Demand: 0.6 -> ₹ 80000
  Low Demand : 0.4 -> ₹ -20000

Product B
  High Demand: 0.7 -> ₹ 60000
  Low Demand : 0.3 -> ₹ 10000

EXPECTED MONETARY VALUE
========================
Expected Value of Product A: ₹ 40000.0
Expected Value of Product B: ₹ 45000.0

Optimal Decision: Select Product B
PS D:\business analytics>
```



Business Insights / Conclusion:

Product A has an expected monetary value of ₹40,000 (0.6 × 80,000 + 0.4 × −20,000), while Product B has ₹45,000 (0.7 × 60,000 + 0.3 × 10,000). Product B is therefore the optimal decision.

It has the higher expected profit and also avoids the risk of a loss, because its worst-case outcome is still a profit of ₹10,000.

Practical 7A: Survey and Sampling

Aim: To conduct a survey, collect data from a sample population using appropriate sampling techniques, and summarize the collected data for business analysis.

Theory: *Sampling selects a representative subset of a population to estimate its characteristics. Simple random sampling gives every member an equal chance of selection; here 50 students are drawn from a population of 1,000 and their monthly food spending is summarized.*

Pract7a.py

```
import numpy as np
import pandas as pd
import matplotlib.pyplot as plt

# Set seed for reproducibility
np.random.seed(43)

# Population of 1000 students
population = np.random.normal(
    loc=3500,
    scale=800,
    size=1000
)
print(population[1:200])

# Select a random sample of 50 students
sample = np.random.choice(
    population,
    size=50,
    replace=False
)

# Create DataFrame
survey data = pd.DataFrame({
    "Student_ID": range(1, 51),
    "Monthly_Food_Spending": np.round(sample, 2)
})

# Display first 10 records
print("Survey Data:")
print(survey_data.head(10))

# Descriptive statistics
print("\nDescriptive Statistics:")
print(survey_data["Monthly_Food_Spending"].describe())

# Calculate Sample Statistics
# Calculate sample mean
sample mean = sample.mean()

# Calculate sample standard deviation
sample_std = sample.std(ddof=1)

print("Sample Mean:", round(sample mean, 2))
print("Sample Standard Deviation:", round(sample std, 2))
```

Output:

```
[2773.21485378 3197.19751515 3072.06752098 4186.45867686 3169.59201415
 3898.55086759 5108.15939806 4510.28923561 3148.62811451 3222.84968561
 3864.25572765 2165.06983439 2810.33160399 3894.3286785  3400.54928308
 5048.10902992 3005.24587601 2662.52881103 2788.30592907 3511.23242976
 3371.33625107 5284.28771907 3180.70742518 3543.55565069 4207.34545216
 3413.61555093 3944.4855874  3815.92530876 4169.76401812 2373.69746368
 4146.27953091 3389.37308429 3649.74286827 3190.67349159 4827.23898383
 1862.34469513 4619.45358904 2956.79430304 4723.18810768 4476.9727706
 4311.99081343 4162.50398588 5313.03416657 3024.03546389 3034.98436998
 2975.28468305 4240.11908269 2460.6709246  4308.93349377 3269.24785379
 2645.82954772 2637.91928195 2862.58098958 2311.16593451 3911.30301254
 4181.43269155 4266.93874982 2998.8127607  3746.3448103  3504.16454795
 4053.22552611 3855.88972726 3572.22362599 2013.32568053 3366.73596486
 3588.70118411 2944.18188479 3284.65954725 2460.61987999 3243.11564232
```

```
 3904.69499373 5171.24765696 2689.83259616 3480.82074378 2730.82475997
 3425.94704883 3321.01433509 4166.31372968 4279.29566271 3630.25452616
 3408.40638392 4449.17474609 3643.83332069 4713.15329411 2192.7743843
 4925.5767208  3005.81783929 2630.88056571 3464.1973376  2569.0380642
 3955.151596   3129.88491557 3729.88408387 2718.12903133 3322.05099325
 3577.5454143  3349.2925209  4773.2817267  3968.64866182 3662.94031361
 2014.78343968 3802.55156494 4488.91225214 2948.02255717 3341.31624623
 1079.40810166 2503.49902127 1832.76842279 3583.29731703 3379.81320474
 3415.34679751 3150.43668882 4315.30905301 2513.75741894 3440.70036936
 3977.1113462  3356.48595029 3427.36394578 2839.93629687 3232.02443112
 4074.68064601 1884.43025226 3373.57676078 3032.62789754 3443.75480183
 4918.54498402 3142.69656593 4691.49966289 4380.2469771  3523.91705436
 3569.68464873 3644.98795375 2839.86256225 4001.47101352 3009.84172232
 3528.59386841 1743.22991081 2809.37384854 3928.07949043 2601.22936383
 3767.17207418 3094.21563583 4292.1201369  4506.3551751  3642.09993497
 3488.10606683 3532.34889146 3482.84418428 4453.3252664  5503.89174824
 2433.23109858 2677.38187705 4590.53573127 2586.18032521 2402.37962807
 5192.29714841 3280.5800916  3299.88366133 3092.93568005 3965.07258353
 2593.51993551 4919.51137967 4102.4370592  3458.55981459 3058.63552043
 3586.60555205 4335.17572114 5593.95410291 4707.01469949 4310.32198966
 3534.43659244 4486.00368851 3222.87132725 3765.50670675 4436.55674271
 3985.41895337 2968.24560579 2273.78090112 3901.19728772 3607.7091517
 4243.99181485 3546.31156941 3196.33309317 4343.16248304 4297.05068247
 1812.64526557 2284.70751613 3603.86770046 3447.80970579 3350.41196015
 4159.14599859 5112.39873197 2674.84149336 3445.98577898 3479.69475328
 3664.23455587 2521.08821517 3873.94390267 3529.36081256]
Survey Data:
   Student ID  Monthly Food Spending
0           1                2013.33
1           2                2478.21
2           3                4259.88
3           4                3765.51
4           5                3944.38
5           6                3379.81
6           7                2634.15
7           8                3130.41
8           9                2379.79
9          10                2628.83

Descriptive Statistics:
count      50.000000
mean     3442.501200
std       834.153902
min      1743.230000
25%      2922.097500
50%      3455.095000
75%      3944.130000
max      5827.630000
Name: Monthly_Food_Spending, dtype: float64
Sample Mean: 3442.5
Sample Standard Deviation: 834.15
```

Business Insights / Conclusion:

A random sample of 50 students from a population of 1,000 gives a mean monthly food spending of ₹3,442.50 with a standard deviation of ₹834.15 (minimum ₹1,743.23, maximum ₹5,827.63). Because every student had an equal chance of selection, the sample is representative and its mean is a good point estimate of the population mean (generated with a mean of ₹3,500).

Such survey data can guide canteen pricing and student-budget decisions.

Practical 7B: Central Limit Theorem

Aim: To apply the Central Limit Theorem (CLT) to analyze the sampling distribution of a sample mean and estimate population parameters.

Theory: *The Central Limit Theorem states that the distribution of sample means becomes approximately normal for large samples, even if the population is skewed, with standard error ≈ σ/√n. 1,000 samples of size 300 are drawn from an exponential population to demonstrate this.*

Pract7b.py

```
import numpy as np
import matplotlib.pyplot as plt

# Set seed
np.random.seed(42)

# Create population
population = np.random.exponential(
    scale=3500,
    size=100000
)

# Sample size
sample_size = 300

# Number of samples
number of samples = 1000

# Store sample means
sample_means = []

# Generate samples
for i in range(number of samples):

    sample = np.random.choice(
        population,
        size=sample size,
        replace=False
    )

    sample_means.append(sample.mean())

sample_means = np.array(sample_means)

# Calculate statistics
population_mean = population.mean()
sampling_mean = sample_means.mean()
standard_error = sample_means.std(ddof=1)

print("Population Mean:", round(population mean, 2))
print("Mean of Sample Means:", round(sampling_mean, 2))
print("Standard Error:", round(standard_error, 2))

# Visualize the Sampling Distribution
plt.figure(figsize=(8, 5))

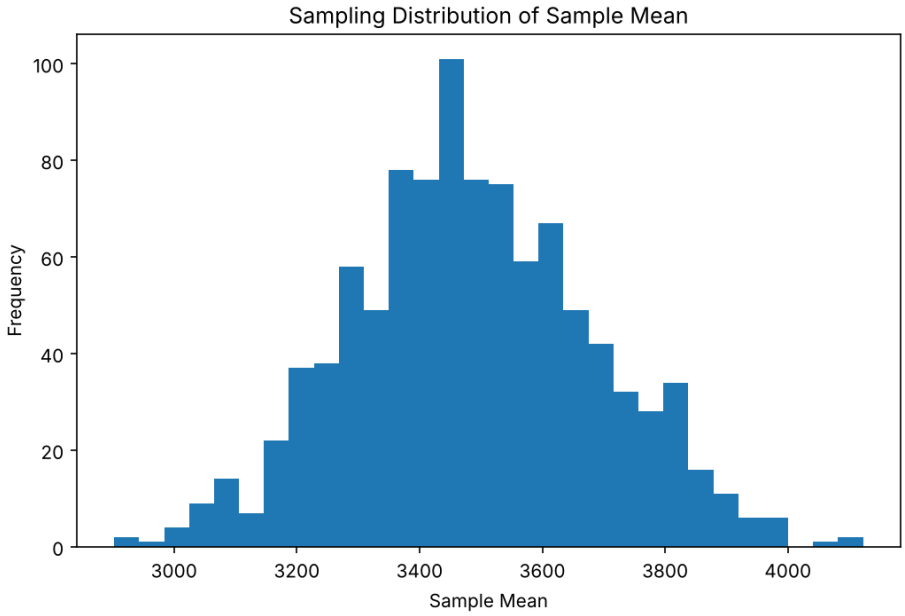
plt.hist(
    sample_means,
    bins=30
)

plt.title("Sampling Distribution of Sample Mean")
plt.xlabel("Sample Mean")
plt.ylabel("Frequency")

plt.show()
```

Output:

```
Population Mean: 3485.9
Mean of Sample Means: 3486.93
Standard Error: 199.71
```



Business Insights / Conclusion:

The population mean is ₹3,485.90 and the mean of the 1,000 sample means is ₹3,486.93, which are almost identical, with a standard error of ₹199.71. Although the exponential population is strongly right-skewed, the histogram of sample means is approximately bell-shaped, which verifies the Central Limit Theorem.

This allows analysts to estimate population averages and build confidence intervals from sample data.

Practical 8A: Confidence Intervals for Mean and Proportion

Aim: To calculate confidence intervals for population means and proportions using sample data and interpret the results in a business context.

Theory: *A confidence interval gives a range likely to contain the population parameter. For a mean with known σ, CI = $\bar{x} \pm z \cdot \sigma/\sqrt{n}$; for a proportion, CI = $p \pm z \cdot \sqrt{(p(1-p)/n)}$, using z = 1.96 for 95% confidence.*

Pract8a.py

```
# confidence intervals for population means

import math

# Sample information
sample_mean = 3520
population_std = 800
sample_size = 50

# Z-value for 95% confidence level
z = 1.96

# Calculate standard error
standard_error = population_std / math.sqrt(sample_size)

# Calculate margin of error
margin of error = z * standard error

# Calculate confidence interval
lower_limit = sample_mean - margin_of_error
upper_limit = sample_mean + margin_of_error

print("Sample Mean:", sample mean)
print("Standard Error:", round(standard error, 2))
print("Margin of Error:", round(margin_of_error, 2))

print(
    "95% Confidence Interval:",
    round(lower_limit, 2),
    "to",
    round(upper_limit, 2)
)



# confidence intervals for proportions

# Survey information
n = 500
satisfied = 325

# Sample proportion
p = satisfied / n

# Z-value for 95% confidence
z = 1.96

# Standard error
standard_error = math.sqrt(
    p * (1 - p) / n
)

# Margin of error
margin_of_error = z * standard_error

# Confidence interval
lower = p - margin_of_error
upper = p + margin_of_error

print("Sample Proportion:", round(p, 4))
print("Sample Proportion (%):", round(p * 100, 2))

print("Standard Error:", round(standard_error, 4))
print("Margin of Error:", round(margin_of_error, 4))
```

```
print(
    "95% Confidence Interval:",
    round(lower * 100, 2),
    "% to",
    round(upper * 100, 2),
    "%"
)
```

Output:

```
Sample Mean: 3520
Standard Error: 113.14
Margin of Error: 221.75
95% Confidence Interval: 3298.25 to 3741.75
Sample Proportion: 0.65
Sample Proportion (%): 65.0
Standard Error: 0.0213
Margin of Error: 0.0418
95% Confidence Interval: 60.82 % to 69.18 %
```

Business Insights / Conclusion:

With a sample mean of ₹3,520, σ = ₹800 and n = 50, the standard error is ₹113.14 and the 95% confidence interval for the population mean is ₹3,298.25 to ₹3,741.75.

In the survey of 500 customers, 65% were satisfied, and the 95% confidence interval for the true satisfaction proportion is 60.82% to 69.18%. The business can be 95% confident that the true values lie within these ranges.

Practical 8B: Bootstrap Confidence Interval

Aim: To apply bootstrapping to estimate confidence intervals for non-parametric statistics such as the median.

Theory: *Bootstrapping estimates the sampling distribution of a statistic by repeatedly resampling the data with replacement. The 2.5th and 97.5th percentiles of 10,000 bootstrap medians give a 95% confidence interval without assuming a normal distribution.*

Pract8b.py

```
import numpy as np

# Original sample
data = np.array([
    450, 520, 600, 480, 750,
    900, 550, 620, 1500, 680,
    720, 500, 580, 810, 650
])

# Number of bootstrap samples
B = 10000

# Store bootstrap medians
bootstrap_medians = []

# Set seed for reproducibility
np.random.seed(42)

# Bootstrap procedure
for i in range(B):

    # Sample with replacement
    bootstrap sample = np.random.choice(
        data,
        size=len(data),
        replace=True
    )

    # Calculate median
    median_value = np.median(bootstrap_sample)

    bootstrap_medians.append(median_value)

# Convert to NumPy array
bootstrap_medians = np.array(bootstrap_medians)

# Calculate original sample median
original_median = np.median(data)

# Calculate 95% confidence interval
lower = np.percentile(
    bootstrap_medians,
    2.5
)

upper = np.percentile(
    bootstrap_medians,
    97.5
)

print("Original Sample Median:", original_median)

print(
    "95% Bootstrap Confidence Interval:",
    lower,
    "to",
    upper
)
```

Output:

```
Original Sample Median: 620.0
95% Bootstrap Confidence Interval: 550.0 to 750.0
```

Business Insights / Conclusion:

The original sample median is ₹620 and the 95% bootstrap confidence interval for the median is ₹550 to ₹750. The sample contains an extreme value (₹1,500), so the median is more reliable than the mean, and bootstrapping gives an interval without assuming any distribution.

This is useful for skewed transaction data.

Practical 9A: Hypothesis Testing

Aim: To formulate null and alternative hypotheses, perform an appropriate statistical hypothesis test using sample data, and interpret the results in a business context.

Theory: *Hypothesis testing uses sample data to decide between the null hypothesis (H0) and the alternative (H1) at significance level α = 0.05. A one-sample t-test checks whether mean sales after a new marketing strategy exceed ₹50,000 (H0: μ = 50,000, H1: μ > 50,000).*

Pract9a.py

```
import numpy as np
from scipy import stats

# Sales data after implementing the new marketing strategy
sales = np.array([
    52000, 54000, 51000, 55000, 53000,
    56000, 52000, 57000, 54000, 55000,
    53000, 58000, 51000, 56000, 54000,
    57000, 55000, 53000, 59000, 56000
])

# Hypothesized population mean
population_mean = 50000

# Perform one-sample t-test
t statistic, p value two tailed = stats.ttest 1samp(
    sales,
    population_mean
)

# Convert two-tailed p-value to one-tailed p-value
if t_statistic > 0:
    p value = p value two tailed / 2
else:
    p_value = 1 - (p_value_two_tailed / 2)

print("Sample Mean:", round(np.mean(sales), 2))
print("t-statistic:", round(t statistic, 4))
print("One-tailed p-value:", round(p_value, 4))

alpha = 0.05

if p_value < alpha:
    print("Decision: Reject H0")
    print("Conclusion: There is significant evidence that sales increased.")
else:
    print("Decision: Fail to reject H0")
    print("Conclusion: There is insufficient evidence that sales increased.")
```

Output:

```
Sample Mean: 54550.0
t-statistic: 9.008
One-tailed p-value: 0.0
Decision: Reject H0
Conclusion: There is significant evidence that sales increased.
```

Business Insights / Conclusion:

The sample mean sales of ₹54,550 give a t-statistic of 9.008 and a one-tailed p-value far below 0.05 (printed as 0.0), so H0 is rejected.

There is significant evidence that average sales increased above ₹50,000 after the new marketing strategy, so the company can justify continuing and extending it.

Practical 9B: A/B Testing

Aim: To perform A/B testing on a website or marketing campaign to evaluate the effectiveness of two different strategies and make a data-driven business decision.

Theory: *A/B testing compares a control (Version A) and a treatment (Version B) using randomly assigned users. A two-proportion z-test checks whether the difference in conversion rates is statistically significant (H0: pA = pB, H1: pA ≠ pB) at α = 0.05.*

A/B Test Program

```
from statsmodels.stats.proportion import proportions_ztest

# Version A
visitors_A = 10000
conversions_A = 500

# Version B
visitors_B = 10000
conversions_B = 600

# Calculate conversion rates
rate A = conversions A / visitors A
rate_B = conversions_B / visitors_B

print("Version A Conversion Rate:",
      rate A * 100, "%")

print("Version B Conversion Rate:",
      rate_B * 100, "%")

# Number of successes
successes = [conversions_A, conversions_B]

# Number of observations
observations = [visitors_A, visitors_B]

# Perform two-proportion z-test
z statistic, p value = proportions ztest(
    successes,
    observations
)

print("\nZ-statistic:", round(z_statistic, 4))
print("P-value:", round(p_value, 4))

# Decision
alpha = 0.05

if p value < alpha:
    print("\nDecision: Reject H0")
    print("Conclusion: There is a statistically significant")
    print("difference between Version A and Version B.")
else:
    print("\nDecision: Fail to reject H0")
    print("Conclusion: There is insufficient evidence")
    print("of a significant difference.")
```

Output:

```
Version A Conversion Rate: 5.0 %
Version B Conversion Rate: 6.0 %

Z-statistic: -3.1016
P-value: 0.0019

Decision: Reject H0
Conclusion: There is a statistically significant
difference between Version A and Version B.
```

Business Insights / Conclusion:

Version A converts at 5% and Version B at 6%. The two-proportion z-test gives $z = -3.10$ and $p = 0.0019$, which is below 0.05, so H0 is rejected and the difference is statistically significant.

Version B improves the conversion rate by 1 percentage point (a 20% relative increase), so the new design should be adopted after considering its implementation cost.

Practical 10A: Regression for Sales Prediction

Aim: To develop a regression model to predict future sales based on historical data, assess the performance of the model, and interpret the significance of predictor variables.

Theory: *Regression predicts a dependent variable from independent variables using $Y = \beta 0 + \beta 1X + \varepsilon$. Model quality is assessed with R^2, MAE and RMSE, and the p-value of the coefficient (OLS) shows whether advertising spend is a statistically significant predictor of sales.*

Regression Program

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
from sklearn.linear_model import LinearRegression
from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error,
    r2_score
)

# Create dataset
data = {
    "Advertising": [10, 12, 15, 18, 20, 22,
                    25, 28, 30, 32, 35, 38],

    "Sales": [50, 54, 60, 65, 70, 73,
              80, 85, 90, 94, 100, 105]
}

df = pd.DataFrame(data)

print("Historical Sales Data")
print(df)

# Independent variable
X = df[["Advertising"]]

# Dependent variable
y = df["Sales"]

# Create regression model
model = LinearRegression()

# Train model
model.fit(X, y)

# Display coefficients
print("\nRegression Coefficients")
print("Intercept:", round(model.intercept , 2))
print("Advertising Coefficient:",
      round(model.coef_[0], 2))

# Predictions
df["Predicted Sales"] = model.predict(X)

print("\nActual vs Predicted Sales")
print(df)

# Predict sales for advertising expenditure of ₹40,000
future_advertising = [[40]]

predicted_sales = model.predict(future_advertising)

print(
    "Predicted Sales for ₹40,000 Advertising:",
    round(predicted sales[0], 2),
    "thousand"
)

# Model predictions
predictions = model.predict(X)

# Calculate performance metrics
mae = mean_absolute_error(y, predictions)
```

```
mse = mean_squared_error(y, predictions)
rmse = np.sqrt(mse)
r2 = r2_score(y, predictions)

print("MODEL PERFORMANCE")
print("------------------")
print("MAE :", round(mae, 2))
print("MSE :", round(mse, 2))
print("RMSE:", round(rmse, 2))
print("R²  :", round(r2, 4))

import statsmodels.api as sm

# Add constant for intercept
X_stats = sm.add_constant(X)

# Create OLS model
ols_model = sm.OLS(y, X_stats).fit()

# Display statistical summary
print(ols_model.summary())

plt.figure(figsize=(8, 5))

plt.scatter(
    df["Advertising"],
    df["Sales"],
    label="Actual Sales"
)

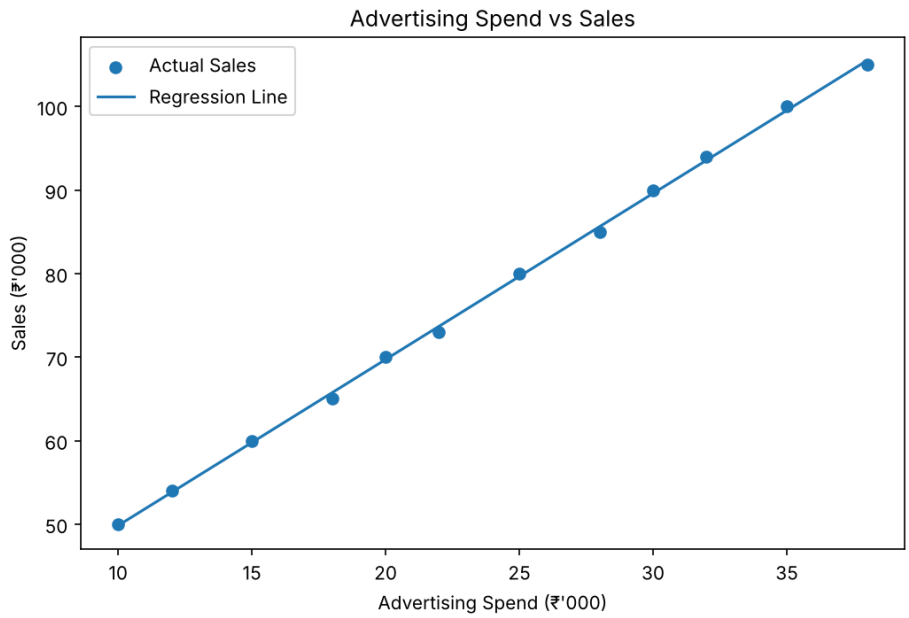
plt.plot(
    df["Advertising"],
    df["Predicted_Sales"],
    label="Regression Line"
)

plt.title("Advertising Spend vs Sales")
plt.xlabel("Advertising Spend (₹'000)")
plt.ylabel("Sales (₹'000)")

plt.legend()
plt.show()
```

Output:

	Advertising	Sales	Predicted_Sales
0	10	50	49.832830
1	12	54	53.808661
2	15	60	59.772407
3	18	65	65.736153
4	20	70	69.711984
5	22	73	73.687815
6	25	80	79.651561
7	28	85	85.615307
8	30	90	89.591138
9	32	94	93.566969
10	35	100	99.530715
11	38	105	105.494461



Business Insights / Conclusion:

The fitted model is Sales = 29.95 + 1.99 × Advertising: every extra ₹1,000 of advertising is associated with about ₹1,990 of additional sales. The model fits the data very well ($R^2 = 0.9993$, MAE = 0.42, RMSE = 0.46) and predicts sales of about ₹109.47 thousand for an advertising spend of ₹40,000.

The p-value of the advertising coefficient in the OLS summary indicates whether the predictor is statistically significant ($p < 0.05$). Regression shows association, not proof that advertising alone causes sales.

Practical 10B: Time Series Forecasting

Aim: To apply time series forecasting techniques, such as exponential smoothing and ARIMA, to forecast future product demand and evaluate forecast accuracy.

Theory: *A time series is a sequence of observations over time. Holt's exponential smoothing weights recent data more heavily and captures trend, while ARIMA(p,d,q) models autocorrelation using differencing; forecast accuracy is judged on held-out test data with MAE and RMSE.*

Forecasting Program

```
import pandas as pd

# Monthly demand data
data = {
    "Month": pd.date_range(
        start="2025-01-01",
        periods=24,
        freq="MS"
    ),

    "Demand": [
        420, 450, 470, 490, 510, 530,
        550, 570, 590, 610, 630, 650,
        670, 690, 710, 730, 750, 770,
        790, 810, 830, 850, 870, 890
    ]
}

df = pd.DataFrame(data)

df.set_index("Month", inplace=True)

print(df)

import matplotlib.pyplot as plt

plt.figure(figsize=(9, 5))

plt.plot(
    df.index,
    df["Demand"],
    marker="o"
)

plt.title("Monthly Product Demand")
plt.xlabel("Month")
plt.ylabel("Demand")

plt.show()

from statsmodels.tsa.holtwinters import ExponentialSmoothing

# Build Holt's model
model_es = ExponentialSmoothing(
    df["Demand"],
    trend="add",
    seasonal=None
)

# Fit model
fit_es = model_es.fit()

# Forecast next 6 months
forecast_es = fit_es.forecast(6)

print("Exponential Smoothing Forecast")
print(forecast_es)

from statsmodels.tsa.arima.model import ARIMA

# Create ARIMA model
model_arima = ARIMA(
    df["Demand"],
    order=(1, 1, 1)
```

```
)

# Fit model
fit_arima = model_arima.fit()

# Forecast next 6 months
forecast_arima = fit_arima.forecast(6)

print("ARIMA Forecast")
print(forecast_arima)

plt.figure(figsize=(10, 5))

# Historical demand
plt.plot(
    df.index,
    df["Demand"],
    label="Historical Demand"
)

# Forecast
forecast_dates = pd.date_range(
    start=df.index[-1] + pd.offsets.MonthBegin(1),
    periods=6,
    freq="MS"
)

plt.plot(
    forecast dates,
    forecast arima,
    marker="o",
    label="ARIMA Forecast"
)

plt.title("Demand Forecast using ARIMA")
plt.xlabel("Month")
plt.ylabel("Demand")

plt.legend()
plt.show()

# Use first 18 months for training
train = df["Demand"].iloc[:18]

# Use last 6 months for testing
test = df["Demand"].iloc[18:]

print("Training Data:")
print(train)

print("\nTest Data:")
print(test)

from statsmodels.tsa.arima.model import ARIMA

# Fit ARIMA on training data
arima model = ARIMA(
    train,
    order=(1, 1, 1)
)

arima_fit = arima_model.fit()

# Forecast test period
arima_predictions = arima_fit.forecast(
    steps=len(test)
)

print("Actual Test Demand:")
print(test.values)

print("\nPredicted Demand:")
print(arima_predictions.values)

from sklearn.metrics import (
    mean_absolute_error,
    mean_squared_error
)
import numpy as np
```

```
mae = mean_absolute_error(
    test,
    arima_predictions
)

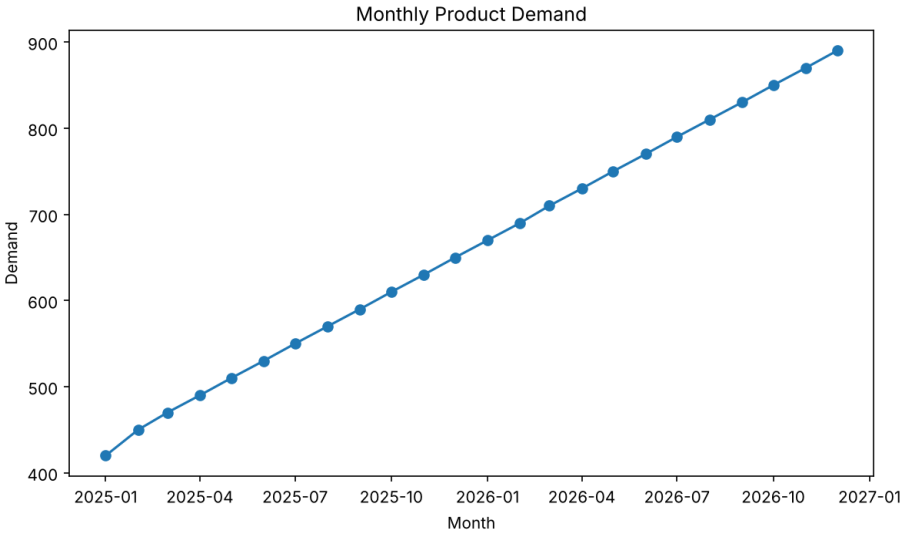
rmse = np.sqrt(
    mean squared error(
        test,
        arima_predictions
    )
)

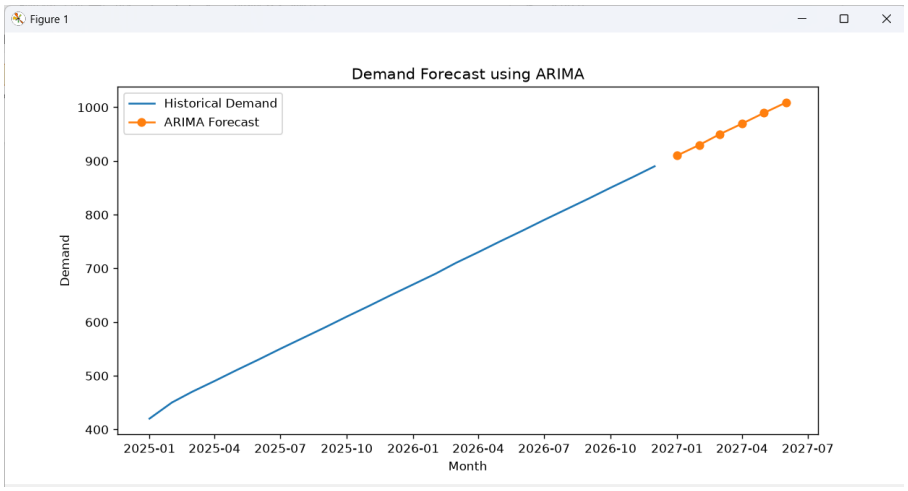
print("ARIMA Forecast Accuracy")
print("-------------------------")
print("MAE :", round(mae, 2))
print("RMSE:", round(rmse, 2))
```

Output:

```
            Demand
Month
2025-01-01     420
2025-02-01     450
2025-03-01     470
2025-04-01     490
2025-05-01     510
2025-06-01     530
2025-07-01     550
2025-08-01     570
2025-09-01     590
2025-10-01     610
2025-11-01     630
2025-12-01     650
2026-01-01     670
2026-02-01     690
2026-03-01     710
2026-04-01     730
2026-05-01     750
2026-06-01     770
2026-07-01     790
2026-08-01     810
2026-09-01     830
2026-10-01     850
2026-11-01     870
2026-12-01     890
```

```
Freq: MS, dtype: float64
c:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueWarning: No frequen
cy information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
c:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueWarning: No frequen
cy information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
c:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueWarning: No frequen
cy information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
c:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\statespace\mlemodel.py:679: EstimationWarning:
Non-stationary starting autoregressive parameters found. Using zeros as starting parameters.
  start_params = self.start_params
ARIMA Forecast
2027-01-01     909.936684
2027-02-01     929.821781
2027-03-01     949.655426
2027-04-01     969.437751
2027-05-01     989.168890
2027-06-01    1008.848975
Freq: MS, Name: predicted_mean, dtype: float64
Training Data:
Month
2025-01-01    420
2025-02-01    450
2025-03-01    470
2025-04-01    490
2025-05-01    510
2025-06-01    530
2025-07-01    550
2025-08-01    570
2025-09-01    590
2025-10-01    610
2025-11-01    630
2025-12-01    650
2026-01-01    670
2026-02-01    690
2026-03-01    710
```





```
2026-06-01    770
Name: Demand, dtype: int64

Test Data:
Month
2026-07-01    790
2026-08-01    810
2026-09-01    830
2026-10-01    850
2026-11-01    870
2026-12-01    890
Name: Demand, dtype: int64
c:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueWarning: No frequen
cy information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
c:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueWarning: No frequen
cy information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
c:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueWarning: No frequen
cy information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
c:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\statespace\mlemodel.py:679: EstimationWarning:
Non-stationary starting autoregressive parameters found. Using zeros as starting parameters.
  start_params = self.start_params
Actual Test Demand:
[790 810 830 850 870 890]

Predicted Demand:
[789.91305637 809.75538162 829.52722697 849.22884278 868.86047849
 888.42238269]
ARIMA Forecast Accuracy
-----------------------
MAE : 0.72
RMSE: 0.88
c:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueWarning: No frequen
cy information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
c:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueWarning: No frequen
cy information was provided, so inferred frequency MS will be used.
```

```
ctual Test Demand:
790 810 830 850 870 890]

redicted Demand:
789.91305637 809.75538162 829.52722697 849.22884278 868.86047849
888.42238269]
RIMA Forecast Accuracy
----------------------
AE : 0.72
MSE: 0.88
:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueW
y information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueW
y information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueW
y information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\base\tsa_model.py:480: ValueW
y information was provided, so inferred frequency MS will be used.
  self._init_dates(dates, freq)
:\DATA BISNESS\.venv\Lib\site-packages\statsmodels\tsa\statespace\mlemodel.py:679: E
on-stationary starting autoregressive parameters found. Using zeros as starting para
  start_params = self.start_params
                  Model       MAE      RMSE
  Exponential Smoothing  1.549707  1.578406
                  ARIMA  0.715439  0.882514
.venv) PS C:\DATA BISNESS>
```

Business Insights / Conclusion:

Monthly demand rises steadily from 420 to 890 over 24 months, which is a clear upward trend. Holt's exponential smoothing and ARIMA(1,1,1) both capture this trend to forecast the next six months.

Training on the first 18 months and testing on the last 6 allows the forecast accuracy to be judged with MAE and RMSE (lower is better). Reliable forecasts help the business plan inventory and production.